Neurocomputing 676 (2026) 132964

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

DyTimeNet: Dynamic cross-variable dependency network with sparse strategy for multivariate time series forecasting

Ting Chen [a, b], Jinzhou Lai [c], Yining Sun [a, b], Wai Kin (Victor) Chan [a, b, *]

[a] Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen, 518055, China
[b] Pengcheng Laboratory, Shenzhen, 518055, China
[c] Hong Kong Polytechnic University, Hong Kong Special Administrative Region of China

HIGHLIGHTS

- A dual-path learning framework integrating time-variable information.
- Novel DropFusion sparse strategy for dynamic cross-variable dependency.
- Strong fine-grained fitting ability using auxiliary variable fusion.
- 7x faster training and superior performance compared to existing methods.

ARTICLE INFO

Communicated by P. Li

Keywords:
Multivariate time series
Cross-variable correlation
Dynamic dependency
Sparse strategy
Neural network

ABSTRACT

Most current research on multivariate time series focuses on extracting temporal patterns, often treating the data as univariate and neglecting interactions between channels. This work introduces variable-wise auxiliary information to facilitate forecasting. In reality, channel relationships are dynamic and sparse, with changing correlations over time, and fusing all channels may distort the underlying data structure. To address this, we propose **DyTimeNet**, a novel dual-path learning framework designed to capture dynamic sparse correlations between variables (CrossVBlock) and complex nonlinear relationships within time series (IntraVBlock). Specifically, we introduce an innovative DropFusion sparse strategy within CrossVBlock, which simulates multiple variable-wise correlations by dynamically selecting connections (edges) and dropping neurons. By doing so, it reduces unnecessary interference from irrelevant variables, forcing the network to avoid over-relying on specific edges or neurons, thereby adapting to varying patterns of channel correlations. Experiments on thirteen real-world datasets demonstrate that DyTimeNet significantly outperforms sophisticated Transformer-based models and even surpasses LLM-based models, especially in capturing better fine-grained pattern fitting in multivariate time series. Additionally, DyTimeNet uses only 1/4 of the parameter size and trains 7× faster than state-of-the-art models like iTransformer, while improving MSE by 10.13%. These improvements make DyTimeNet ideal for applications with limited computational resources or high-frequency demands, such as edge-side sensor networks.

1. Introduction

Multivariate time series data are prevalent in various fields, including economics, meteorology, medicine, engineering, and environmental science. A typical characteristic of this type of data is the presence of complex inter-dependencies among different variables, which may evolve over time and exhibit distinct nonlinear features. For example, Fig. 1 shows Pearson correlation matrices of traffic flows from 100 sensors[1] at city intersections. The correlation structure changes markedly across the morning (Fig. 1a), noon (Fig. 1b), and evening (Fig. 1c) periods. These dynamic changes could be influenced by factors such as traffic flow, congestion, pedestrian movement, or other environmental conditions. Beyond transportation, meteorological variables (e.g., temperature, humidity, wind speed) interact in regime-dependent ways, yielding different correlation structures at different times of day

* Corresponding author at: Tsinghua Shenzhen International Graduate School, Tsinghua University, Shenzhen, 518055, China.
Email address: chanw@sz.tsinghua.edu.cn (W.K. (Victor) Chan).

[1] In this paper, the terms 'sensors', 'variable', 'variate', and 'channel' all refer to the same concept: the individual variables being measured.

https://doi.org/10.1016/j.neucom.2026.132964
Received 20 September 2025; Received in revised form 21 December 2025; Accepted 3 February 2026
Available online 10 February 2026
0925-2312/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

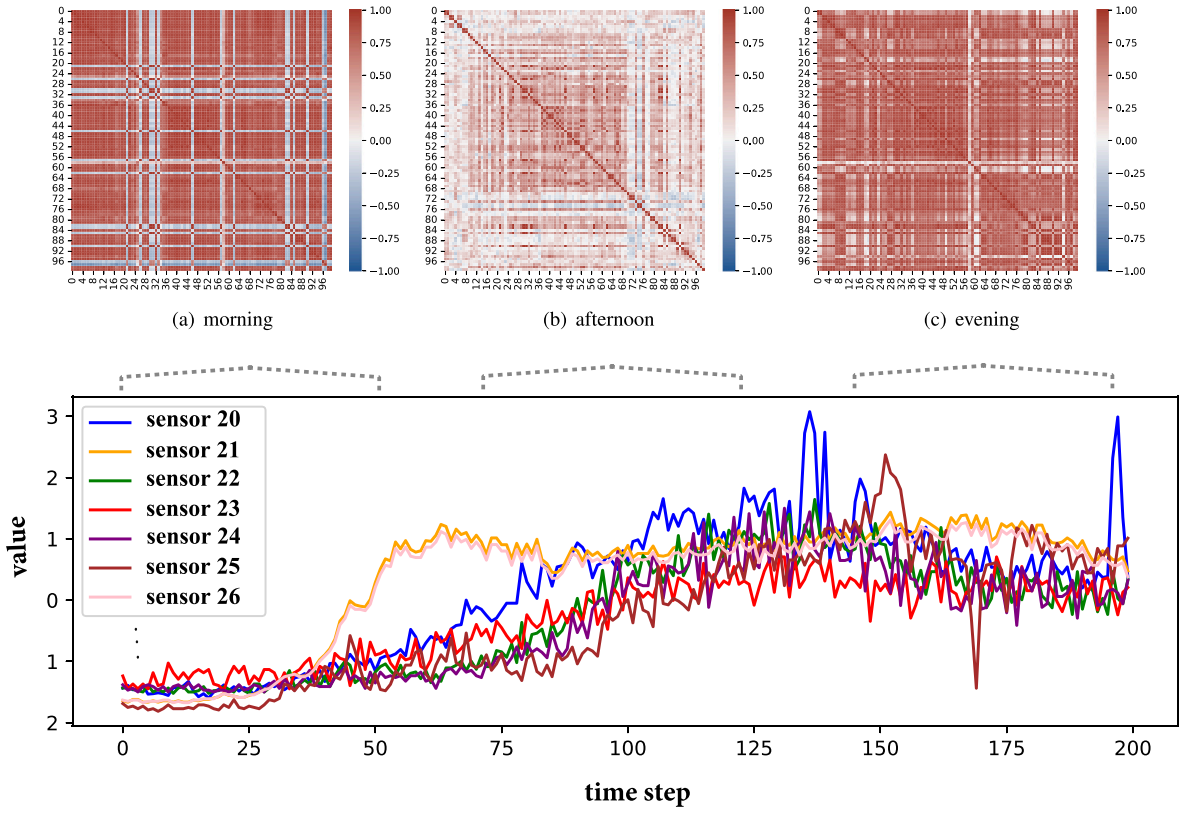


Fig. 1. Dynamic Uncertainty in Multivariate Correlation (PEMS03, a traffic dataset): In (d), the actual traffic flow recorded by a subset of sensors/variables (20–26) over 24 hours is shown. In (a)–(c), the corresponding Pearson Correlation Coefficient heatmaps for all sensors across different periods are presented, revealing the time-varying correlations and dynamic patterns between variables.

or under distinct synoptic conditions. Similarly, in financial markets, the relationships among stock prices, interest rates, and trading volumes are dynamic and filled with nonlinear characteristics.

However, most current research assumes independence between channels [1–3], and then mainly targets the capture of temporal variations such as periodicity, fluctuations, and trends. Among them, Transformer-based methods, which utilize the Attention mechanism to uncover internal dependencies within time series, have shown significant promise. However, recent works [2,4] revisit the effectiveness of self-Attention by showing that a simple one-layer linear model can outperform complex Transformer-based models in capturing temporal relationships, highlighting the potential temporal information loss in Attention mechanisms. Subsequently, an increasing number of MLP-based methods have shown superior performance compared to Transformer-based models, such as TimeMixer [5], TSMixer [6], Fits [7], MPMixer [8], among others.

Therefore, **it is necessary to reconsider the necessity of the complex designs in existing Transformer-based methods**. As shown in Table 1, contrary to typical expectations, removing the Attention mechanism **(w/o Attention)** from Transformer-based models like Transformer, Informer, and PatchTST actually improves performance (lower MSE and MAE) in most cases, with a maximum improvement of 24.31%. This indicates that the Attention mechanism may introduce unnecessary complexity or fail to capture relevant patterns effectively. Instead, simpler MLP models (i.e., FFN) might better extract temporal patterns in univariate time series. The likely explanation is that the inherent nonlinear characteristics of time series, such as periodic fluctuations, abrupt changes, and long-tail dependencies, are challenging for the standard Attention mechanism to capture effectively.

Furthermore, regarding the issue of channel dependencies, existing studies typically assume that the relationships between channels are either fixed [9,10] or fully correlated [5,11,12]. However, in reality, the relationships between channels in multivariate time series should exhibit dynamic behavior and may not always be fully correlated. As clearly shown in Fig. 1, the dynamic and sparse nature of channel relationships is evident. In different time periods, the relationships between channels can vary significantly, for example, sensor A and sensor B may exhibit a positive correlation in one time segment, but show no correlation, or even a negative correlation, in subsequent periods.

Motivated by this, **we need to amplify the role of MLP**, and **rethink how to make MLP better adapt to the characteristics of time series**, and **how to address the cross-variable correlations in multivariate time series?** To this end, we propose the DyTimeNet method, a dual-level FFN framework designed to capture complex nonlinear relationships within time series and the dynamic sparse correlations between variables. The key contributions of this work include:

- The introduction of DyTimeNet, a dual-level time-variable module designed to model both temporal and partial cross-channel dependencies with a novel sparse strategy, thereby ensuring better generalization and robustness in both stationary and non-stationary time series.
- To model dynamic and sparse cross-variable dependencies, we propose the Cross-Variable Block (CrossVBlock) equipped with an adaptive DropFusion strategy, which dynamically selects connections and eliminates neurons during learning process, thereby reducing interference from unrelated variables and adapting to varying patterns of channel correlations.

Table 1
The effectiveness of FFN (i.e., MLP) for temporal modeling. Comparison of Transformer-based methods **(w & w/o Attention)**. All the results are averaged across 4 different prediction lengths. **IMP(%)**: Improvement Percentage.

Model		Transformer (2017)		Informer (2021)		PatchTST (2023)	
Metric		MSE	MAE	MSE	MAE	MSE	MAE
ETTh2	Original	3.562	1.503	4.431	1.729	0.387	0.407
	w/o Attention	**3.354**	**1.396**	**3.354**	**1.396**	**0.386**	**0.405**
	IMP(%)	**5.85%(↑)**	**7.13%(↑)**	**24.31%(↑)**	**19.29%(↑)**	**0.26%(↑)**	**0.49%(↑)**
Weather	Original	0.657	0.572	0.634	0.548	**0.259**	**0.281**
	w/o Attention	**0.508**	**0.515**	**0.508**	**0.515**	0.264	0.285
	IMP(%)	**22.66%(↑)**	**10.05%(↑)**	**19.85%(↑)**	**6.11%(↑)**	−1.93%	−1.42%
ECL	Original	0.277	0.372	0.311	0.397	0.216	0.304
	w/o Attention	**0.271**	**0.365**	**0.271**	**0.365**	**0.206**	**0.292**
	IMP(%)	**2.26%(↑)**	**2.02%(↑)**	**12.94%(↑)**	**8.19%(↑)**	**4.82%(↑)**	**4.03%(↑)**
Traffic	Original	0.665	0.363	0.764	0.416	0.555	0.362
	w/o Attention	**0.655**	**0.346**	**0.655**	**0.346**	**0.489**	**0.317**
	IMP(%)	**1.47%(↑)**	**4.75%(↑)**	**14.23%(↑)**	**16.89%(↑)**	**11.93%(↑)**	**12.46%(↑)**

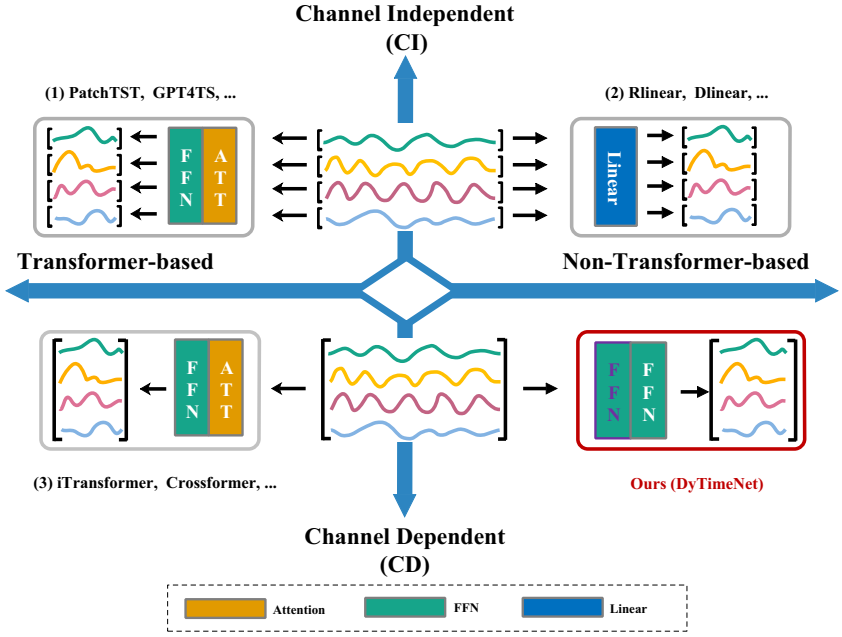


Fig. 2. Multivariate time series forecasters categorized by channel process and model design.

- To model temporal dependencies, we propose Intra-Variable block (IntraVBlock), which captures the complex, nonlinear relationships within individual time series. By selectively fusing information from certain channels, it enhances the model's ability to capture finer-grained temporal features, including multi-cycle patterns, sudden changes, and local fluctuations (e.g., peaks and valleys).
- Moreover, our proposed DyTimeNet consistently outperforms state-of-the-art methods (including current LLM-based approaches) in both long-term and short-term forecasting tasks, while delivering superior efficiency across diverse benchmarks.

2. Related work

We present a brief overview of recent advancements in multivariate time series research, categorized into four areas based on channel processes (Channel Dependent (CD) or Channel Independent (CI)) and model design (Transformer-based or Non-Transformer), as illustrated in Fig. 2.

Transformer-based models for multivariate time series. Transformers have become increasingly popular in time series analysis, primarily due to their strength in understanding sequential dependencies. Categorized by the granularity of representation learning [12], most Transformer-based models explore methods to align time series with textual data, including point-token [13], patch-token [1,3], and series-token [9], and then apply the Attention mechanism to capture the relationships among these temporal tokens. However, most Transformer-based methods primarily target the capture of temporal dependencies, neglecting the inter-channel dependencies. Expanding the receptive field further, methods such as Crossformer [11] and iTransformer [9], as well as TimeXer [12], consider the influence of exogenous variables, utilizing the Attention mechanism to learn multivariate correlations, thereby enhancing temporal expression capabilities and demonstrating superior forecasting performance. Additionally, models such as CVACL-MA [14] and DSIN-PMA [15] build upon the Transformer framework by further refining channel correlations to mitigate noise and interference from irrelevant channels. These methods employ Pearson correlation coefficients or channel similarity-based partitioning strategies to filter out unrelated channels. Subsequently, the Attention mechanism is used to learn the weights of related channels, thus enabling more robust and accurate representation learning.

LLM-based models for multivariate time series. With the development of large models, some studies have proposed reprogramming large language models (LLM) for time series tasks, which are primarily categorized into LLM4TS [16,17] and TS4LLM [18]. Specifically, LLM4TS primarily focuses on transferring pretrained language models to time series data, while TS4LLM involves modeling large time series based on standard Transformers with more layers from scratch. In summary, current Transformer-based or LLM-based modifications essentially adapt the Attention mechanism to accommodate time series data, including adapting to the fluctuation relationships within time series as well as the interrelation effects between variates. However, these adaptations overlook the fact that Attention fundamentally involves linear weighting computations, while relationships within time series are predominantly nonlinear. Therefore, we argue that using Attention to extract information within time series is inherently contradictory.

MLP-based models for multivariate time series. Compared to the complex Transformer frameworks, MLP methods are simpler and more efficient. Their strong nonlinear regression capability may be better suited for time series forecasting tasks. As a result, some studies [2,4,5,19–21] have begun to reconsider the role of MLPs in time series analysis and question whether the Attention mechanism is truly appropriate for time series. For example, recent works [2,4] revisit the effectiveness of self-Attention by demonstrating that a simple one-layer linear model can outperform complex Transformer-based models in capturing temporal relationships, highlighting the potential temporal information loss in Attention mechanisms. Furthermore, considering cross-channel dependencies, some MLP-Mixing methods, such as MTS-Mixers [22], TS-Mixer [6], and TSMixer [23], which adapt the vision MLP-Mixer [24] for time series analysis, integrate other exogenous variables and have shown promising results. Moreover, some studies [7] focus on designing ultra-lightweight MLP-based models tailored for resource-constrained or edge devices, enabling real-time prediction with minimal computational cost.

Channel Independent & Channel Dependent. There are two main training strategies for the multivariate time series forecasting task [25], as shown in Fig. 2. The top part illustrates the Channel Independent (CI) strategy, represented by methods such as PatchTST [1], GPT4TS [3], and Rlinear [2], which treat the multivariate time series as a collection of individual univariate series and adopt a shared backbone for these series. The prediction for each channel depends solely on its own historical values, and the relationships between different channels are ignored. The bottom part illustrates the Channel Dependent (CD) strategy, where all channels are taken as input, and the forecasted future values depend on the history of all channels.

Recently, Channel Dependent (CD) methods [2,11,25,26] have achieved state-of-the-art performance in multivariate time series (MTS) forecasting. Research [12] has found that time series data are often influenced by external factors, such as economic indicators, demographic changes, and societal events. For instance, electricity prices are highly dependent on the supply and demand in the market, and it is nearly impossible to predict future prices based solely on historical data. Incorporating external factors provides a more comprehensive understanding of the interrelationships and causality among various variables, leading to better and more reliable performance, as well as increased interpretability. Moreover, the latest research [27] indicates that considering dependencies between channels can significantly improve the performance of multivariate time series prediction, especially on datasets with strong correlations, compared to methods that assume channel independence. A deeper exploration and balance of channel dependencies provide valuable insights for designing new methods and optimizing the performance of existing ones.

In channel dependency modeling, several researchers [14,15,28,29] explicitly model the relationships between channels, such as similarity computation, weight allocation, or gating mechanisms, combined with graph learning or Attention mechanisms to fuse relevant channel information while effectively filtering out noise and irrelevant channels. These methods primarily focus on fusing linearly correlated channel information. In this work, we explore the powerful nonlinear capabilities of MLPs, particularly their potential for complex channel fusion, including the handling of multiple dynamic relationships such as linear, nonlinear, and independent ones.

Overall, MLP-based approaches are more flexible and practical, particularly in scenarios with limited computational resources, and thus hold significant potential for real-world deployment. However, existing models, whether Transformer-based or MLP-based, which account for temporal and channel dependencies, often overlook a critical property of real-world multivariate time series: channel relationships are dynamically evolving and sparse over time. Ignoring this characteristic may lead to the incorporation of irrelevant channel information, which can degrade forecasting performance. Therefore, this study focuses on effectively modeling the dynamic and sparse nature of channel dependencies to enhance predictive accuracy.

3. Problem statement

Let $X_t = \{x_t^{(1)}, x_t^{(2)}, \dots, x_t^{(C)}\}$ represent the vector of multivariate time series data at time point t, where each $x_t^{(i)}$ is the observed value of the i-th variable at time t, and C denotes the number of variables.

Cross-Variate Dependencies: Variate correlation refers to the potential relationships or interactions between different variables in a multivariate time series. These relationships are often complex and ambiguous, making it challenging to fully characterize them. The correlation between variables can be understood through two key dimensions: Diversity (D) and Dynamic Uncertainty (DU), collectively referred to as the "D-DU" characteristic.

(1) Diversity (D): In a multivariate time series, the nature of the interdependence between any two variables can vary significantly. For example, Variable A and Variable B might exhibit a linear relationship, have no observable correlation, or interact in a nonlinear manner that may be influenced by external factors or other variables. These relationships can generally be categorized into three types: linear, nonlinear, and independent.

(2) Dynamic Uncertainty (DU): This refers to the dynamically changing relationships among channels over time, as illustrated in Fig. 1. The Pearson Correlation Coefficients of traffic flow from 100 sensors exhibit varying correlations at different periods of the day (morning (a), noon (b), and evening (c)). These dynamic changes could be influenced by factors such as traffic flow, congestion, pedestrian movement, or other environmental conditions.

Consequently, during certain periods, the relationship may be well characterized by a model. However, as time progresses, the initial correlation could dissipate, leading to a lack of any apparent relationship in subsequent periods.

Problem Definition. Given the first variable series to be predicted, the historical series is represented as $\mathbf{x}_{1:L} = \mathbf{z}_{1:L}^{(1)} = \left\{x_1^{(1)}, x_2^{(1)}, \dots, x_L^{(1)}\right\} \in \mathbb{R}^{L\times 1}$, along with corresponding inter-series $\mathbf{Z}_{1:L} = \left\{\mathbf{z}_{1:L}^{(2)}, \dots, \mathbf{z}_{1:L}^{(C)}\right\} \in \mathbb{R}^{L\times(C-1)}$, where $\mathbf{z}_{1:L}^{(j)}$ corresponds to the j-th variable with L lookback length, x_L^j represents the value of j-th variable at the L-th time point, and C refers to the total number of variables. The objective of the forecasting model f is to estimate the future S time steps $\hat{\mathbf{y}}_{predict} = \left\{y_{L+1}^{(1)}, y_{L+2}^{(1)}, \dots, y_{L+S}^{(1)}\right\}$ utilizing both the historical observations $\mathbf{x}_{1:L}$ and the inter-series $\mathbf{Z}_{1:L}$. Therefore, the problem is mathematically formulated as follows:

$$\begin{aligned}\hat{\mathbf{y}}_{predict} &= f\left(\mathbf{x}_{1:L}; \mathbf{x}_{1:L} | \mathbf{Z}_{1:L}\right) \\ &= f\left(\mathbf{x}_{1:L}^{(1)}; \mathbf{x}_{1:L}^{(1)} | \left\{\mathbf{z}_{1:L}^{(2)}, \dots, \mathbf{z}_{1:L}^{(C)}\right\}\right).\end{aligned} \tag{1}$$

The objective function is the Mean Squared Error (MSE) Loss:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{S}\sum_{t=1}^{S}\left(x_{L+t}^{(1)} - \hat{x}_{L+t}^{(1)}\right)^2 \tag{2}$$

4. Preliminary

4.1. Time-varying autoregression on multivariate data

In the context of multivariate time series, as shown in Eq. (3), the data are represented in a $C \times L$ matrix form, where L denotes the length of historical data and C represents the number of channels. In statistics, Vector Autoregression (VAR) is a classical method widely used to characterize the co-evolution patterns of multiple variables in time series data [30]. Each coefficient in the VAR model is explained by its own past values and the past values of other variables in the real-world time-varying system.

$$\mathbf{X}_{1:L} = \begin{bmatrix} x_{t-1}^{(1)} & x_{t-2}^{(1)} & \cdots & x_{t-L}^{(1)} \\ x_{t-1}^{(2)} & x_{t-2}^{(2)} & \cdots & x_{t-L}^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_{t-1}^{(C)} & x_{t-2}^{(C)} & \cdots & x_{t-L}^{(C)} \end{bmatrix} = \{X_{t-1}, X_{t-2}, \ldots, X_{t-L}\} \tag{3}$$

Specifically, to characterize the time-varying system behavior, the VAR model considers the time-varying coefficient matrices $A_i \in \mathbb{R}^{C\times C}$, $\forall i \in [L]$ in the vector autoregression [31,32], namely,

$$\begin{aligned} X_t &= A_1 X_{t-1} + A_2 X_{t-2} + \cdots + A_L X_{t-L} + \epsilon_t \\ &= \sum_{i=1}^{L} A_i X_{t-i} + \epsilon_t \end{aligned} \tag{4}$$

where $X_t \in \mathbb{R}^{C\times 1}$ is the vector of C variables at time t, $A_1, A_2, \ldots, A_L \in \mathbb{R}^{C\times C}$ are the coefficient matrices that capture the relationships between the variables at different lags (from $t-1$ to $t-L$), and $\epsilon_t \in \mathbb{R}^{n\times 1}$ is the error term, assumed to be white noise. The coefficient matrices A_i can be represented as a tensor with $C \times C$ coefficients $a_{jk}^{(i)}$, where each coefficient $a_{jk}^{(i)}$ represents the impact of the j-th variable at time $t-i$ on the k-th variable at time t at lag i:

$$A_i = \begin{bmatrix} a_{11}^{(i)} & a_{12}^{(i)} & \cdots & a_{1C}^{(i)} \\ a_{21}^{(i)} & a_{22}^{(i)} & \cdots & a_{2C}^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{C1}^{(i)} & a_{C2}^{(i)} & \cdots & a_{CC}^{(i)}. \end{bmatrix} \tag{5}$$

Mathematically, the coefficient matrices $A_1, A_2, \ldots, A_L$ are typically estimated using Ordinary Least Squares (OLS) or higher-order tensor factorization [33], which minimizes the sum of squared errors between the observed and predicted values. Therefore, the time-varying autoregression can be reformulated as:

$$\begin{cases} X_t = \sum_{i=1}^{L} A_i X_{t-i} + \epsilon_t, & \text{(Autoregression)} \\ (A_1, A_2, \ldots, A_L) = (X_{1:L}^T X_{1:L})^{-1} X_{1:L}^T X_t, & \text{(Ordinary Least Squares)} \end{cases} \tag{6}$$

As a result, the optimization problem for the VAR model can be written as:

$$\min_{A} \sum_{t=L}^{L+S} \left\| X_t - \sum_{i=1}^{L} A_i X_{t-i} \right\|^2 \tag{7}$$

4.2. Neural network for multivariate regression

The time-varying autoregression (VAR) model characterizes temporal dependencies by estimating a coefficient matrix $A \in \mathbb{R}^{C\times C}$ that maps past observations to future values, thereby capturing the dynamic evolution patterns in multivariate time series data. However, while VAR is effective at capturing linear relationships and short-term dynamics, its fixed linear assumptions and simplicity can limit its ability to model more complex, nonlinear temporal dynamics.

In contrast, as illustrated in Fig. 3, neural networks (NNs) offer a flexible alternative to VAR for modeling multivariate time series. In NNs, a learnable weight matrix $\mathcal{W} \in \mathbb{R}^{C\times C}$ maps historical observations to future predictions, serving a similar role as the coefficient matrix $A \in \mathbb{R}^{C\times C}$ in VAR models. Additionally, neural networks incorporate nonlinear activation functions and deeper architectures, which enable them to capture complex, high-dimensional, and nonlinear temporal dependencies. This makes them particularly well-suited for forecasting dynamic systems characterized by intricate or nonstationary behavior. The general form of multivariate regression using neural networks can be expressed as:

$$\begin{aligned} X_t &= f(X_{t-1}, X_{t-2}, \ldots, X_{t-L}) = \sigma\left(\mathcal{W} * X_{t-1:t-L} + \epsilon_t\right), \\ A_i X_{t-i} &\leftarrow \mathcal{W} * X_{t-i}, \end{aligned} \tag{8}$$

where $\sigma(\cdot)$ denotes a nonlinear activation function, $\mathcal{W}$ represents the learnable network parameters, and ϵ_t is the residual error. The parameters $\mathcal{W}$ are optimized via backpropagation, playing a role analogous to estimating the VAR coefficient matrices $\{A_i\}_{i=1}^{L}$, but with enhanced expressive power enabled by nonlinearity and hierarchical representations.

However, while a single-layer neural network can effectively capture short-term dynamics, it may struggle with long-term dependencies due to the issue of catastrophic forgetting [34,35], where the model tends to forget earlier information as it learns new patterns. To address this, we propose a dual-path learning framework, which introduces an additional path to better capture the full spectrum of temporal dynamics within each channel. This approach simultaneously captures both cross-variable interactions and long-term intra-variable dependencies, forming a robust variable-temporal dual-path fusion learning paradigm.

5. Methodology

In terms of multivariate time series forecasting, the future trend for a specific variable depends on its historical data (intra-variable) as well as potential influences from other variables (cross-variable).

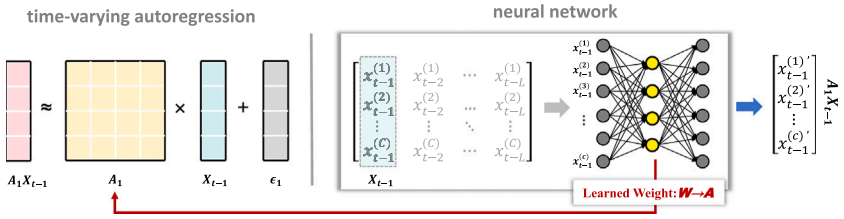


Fig. 3. Time-varying autoregression and neural network for multivariate time series.

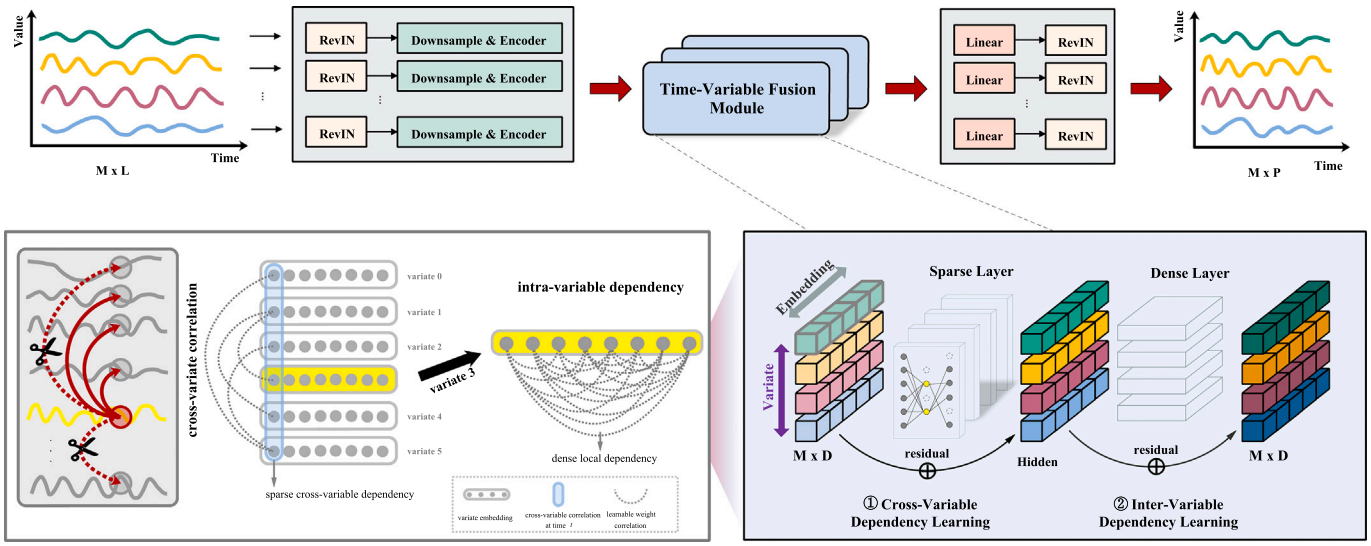


Fig. 4. Overall structure of **DyTimeNet**, featuring a dual-level time-variable module that integrates temporal and cross-channel information. The model uses RevIN layers followed by downsampling and encoding, with key components for cross-variable dependency learning, intra-variable dependency learning, and a sparse strategy, enhancing prediction performance in multivariate time series.

5.1. Overall architecture

As illustrated in Fig. 4, the proposed DyTimeNet framework employs a dual-level Feed-Forward Networks (FFN): an inverted FFN for capturing variate-wise dependencies and a standard FFN for capturing temporal-wise dependencies. Additionally, the DropFusion strategy is introduced to balance the multiple types of variable relationships (linear, nonlinear, and independent). Technically, predicting the future series for each specific variate $\hat{\mathbf{y}}_{predict}$ based on the historical series $\mathbf{x}_{1:L}$ primarily involves three key processes: data preprocessing (including Reversible Instance Normalization (RevIN) [36] and Variate Embedding), model learning (using CrossVBlock & IntraVBlock), and data postprocessing (involving Temporal Projection and Instance Denormalization). Formally, this process is outlined by the equation below:

$$\tilde{\mathbf{x}} = \text{RevIN}(\mathbf{x}_{\mathbf{1:L}}) = \frac{\mathbf{x}_{\mathbf{1:L}} - \mu(\mathbf{x}_{\mathbf{1:L}})}{\delta(\mathbf{x}_{\mathbf{1:L}}) + \varepsilon}, \tag{9}$$

$$\tilde{\tilde{\mathbf{x}}} = Temporal\,Encoding(\tilde{\mathbf{x}}) \rightarrow \mathcal{X} = \{\tilde{\tilde{\mathbf{x}}}^{(1)}, \tilde{\tilde{\mathbf{x}}}^{(2)}, \ldots, \tilde{\tilde{\mathbf{x}}}^{(C)}\}, \tag{10}$$

$$\hat{\mathbf{x}} = IntraV\,Block(CrossV\,Block\,(\mathcal{X})), \tag{11}$$

$$\hat{\hat{\mathbf{x}}} = Projection(\hat{\mathbf{x}}), \tag{12}$$

$$\hat{\mathbf{y}}_{predict} = \hat{\hat{\mathbf{x}}} \cdot (\sigma(\mathbf{x}_{\mathbf{1:L}}) + \varepsilon) + \mu(\mathbf{x}_{\mathbf{1:L}}), \tag{13}$$

where $\mathcal{X} = \{\tilde{\tilde{\mathbf{x}}}^{(1)}, \tilde{\tilde{\mathbf{x}}}^{(2)}, \ldots, \tilde{\tilde{\mathbf{x}}}^{(C)}\}$ denotes the multivariate time series embedding matrix, and ε is a small constant for numerical stability. In this procedure, Reversible Instance Normalization (RevIN) is carried out by normalizing the input individual series $\mathbf{x}_{\mathbf{1:L}}$ by its mean $\mu(\mathbf{x}_{\mathbf{1:L}})$ and standard deviation $\delta(\mathbf{x}_{\mathbf{1:L}})$. Moreover, $Temporal\,Encoding : \mathbb{R}^L \rightarrow \mathbb{R}^D$ and $Projection : \mathbb{R}^D \rightarrow \mathbb{R}^S$ are both implemented by trainable linear projectors. Specifically, considering the time series data with the temporal redundancy [22,37], we apply downsampling to the original time series, thus setting $D < L$ during variate embedding.

5.2. Temporal downsampling and encoding

Current Transformer-based models primarily embed individual time points (point-tokens) or sub-series (patch-tokens) as temporal tokens, and then employ Encoder-only architecture to obtain time series representation. However, due to the high sampling frequency, time series data inherently contain a significant amount of redundant information [38]. As a result, overly fine-grained representations, like point-token or patch-token, amplify irrelevant information, potentially obscuring key information. Therefore, we employ downsampled series-token representations using a trainable linear projector or alternatively a 1D convolutional layer to perform the downsampling operation. The formula is presented below:

$$Temporal\,Encoding\,(\tilde{\mathbf{x}}) = \tilde{\mathbf{x}} * \mathcal{W}_{enc} + \mathbf{b}_{enc}, \tag{14}$$

where $\tilde{\mathbf{x}}$ is the input sequence of the specific variate, $\mathcal{W}_{enc} \in \mathbb{R}^{L \times D} (D < L)$ is the learnable filter, and $\mathbf{b}_{enc}$ is the bias. This neural network learning process helps to capture intra-series patterns (e.g., amplitude, periodicity, and trend) across the entire series while reducing redundant information. This approach ensures that the embedded vector will contain the relevant patterns for accurate time series representation. Additionally, temporal downsampling provides a form of temporal compression, where the sequence length is reduced, and only the most significant temporal features are retained. This compression allows for more efficient downstream processing, reducing computational load and mitigating overfitting.

5.3. CrossVBlock: cross-variable sparse dependency learning

Attention mechanisms in Transformers primarily perform a linear weighted sum, which means they can only capture linear relationships among variables in time series data. To explore more complex relationships, substituting MLPs for the Attention mechanism while retaining the Transformer's architecture will theoretically reveal intricate nonlinear relationships among variables. MLPs, with their hidden layers and nonlinear activation functions, are capable of capturing complex patterns and dependencies that go beyond linear associations. Therefore, as shown in Fig. 5, we invert the standard FFN to better capture variate-wise dependencies across all sequence points, aiding in the identification of underlying Dynamic Uncertainty (DU).

Layer Normalization. Layer normalization ensures consistent feature distributions across variables, stabilizing the training process:

$$\mathcal{X}' = LayerNorm(\mathcal{X}) = \frac{\mathcal{X} - \mu(\mathcal{X})}{\delta(\mathcal{X}) + \varepsilon}, \tag{15}$$

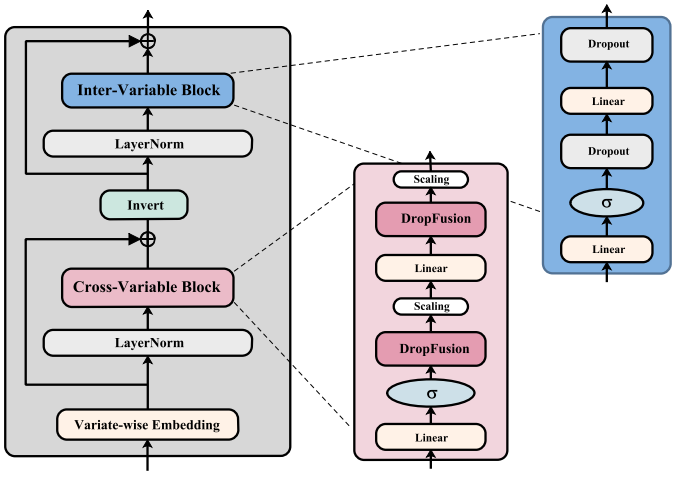


Fig. 5. Time-Variable fusion module architecture.

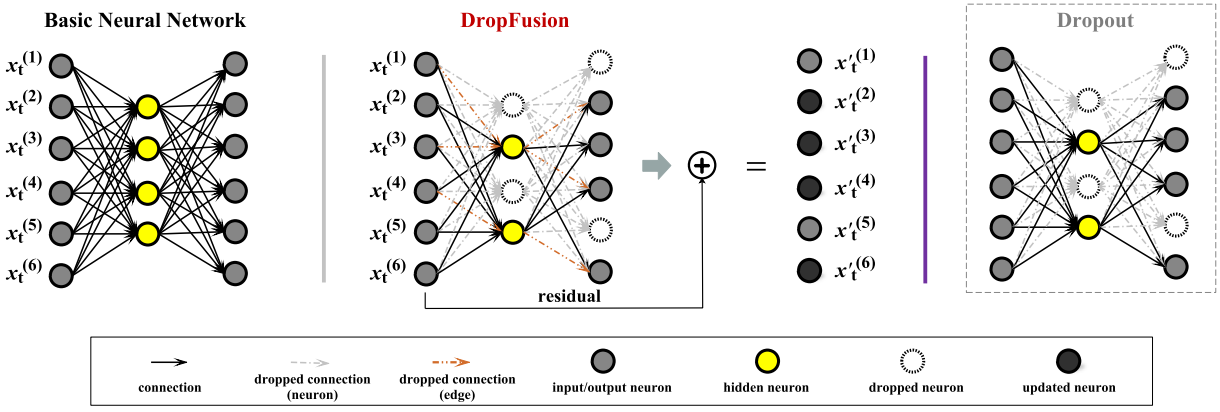


Fig. 6. DropFusion. DropFusion modifies the neural network by dropping some connections and hidden neurons, then uses residual connections to retain and reintroduce crucial information into the output layer. This contrasts with the Dropout approach, which only removes neurons without any mechanism to recover the lost information.

where the normalization is applied to each variable across the sequence length D, and $\mu(\mathcal{X}), \delta(\mathcal{X})$ represent the mean and standard deviation. This normalization is applied before both the Cross-Variable operation and the Intra-Variable Block, ensuring smoother learning.

Cross-Variable Block. We first transpose $\mathcal{X}'$, allowing the model to learn the relationships between the variables. Similarly, we adopt a neural network regression approach to model these relationships. However, unlike the structure of a traditional Multi-Layer Perceptron (MLP), we replace Dropout with DropFusion. This adjustment is driven by the Diversity (D) characteristic of cross-variate dependencies, where some channels exhibit correlation while others may remain independent. Consequently, we propose DropFusion as a method to simulate these complex multivariate correlations by dynamically selecting connections and dropping neurons, as illustrated in Fig. 6, where six variate values at time t correspond to six neurons.

Specifically, during each training step, DropFusion simulates independent variates (e.g., $x_t^{(1)}$, $x_t^{(5)}$) by dropping neurons (dashed circles) and models the absence of relationships between variables by dropping edges (orange dotted line), while integrating residual connections to preserve the information of independent variates. Considering the redundancy observed among variables [22], we set the number of hidden layer neurons C' to be less than the number of variates C (i.e., $C' < C$) to promote variate-centric processing as discussed in [9]. This design forces the neural network to not over-rely on any specific connectivity pattern, adapting to diverse and dynamic variate correlations. Let σ be an activation function. The CrossVBlock process will then be

$$\mathcal{X}'' \leftarrow (\mathcal{X}')^T, \tag{16}$$

$$CrossV\,Block(\mathcal{X}'') = DropFusion\left(\sigma\left(Linear(\mathcal{X}'')\right)\right). \tag{17}$$

Essentially, DropFusion builds on Dropout by adding drop connection to reduce unnecessary interference from unrelated variables. Besides that, by sharing parameters, we hope to learn additional complex relationships among variables, such as causal, lagged, and conditional relationships, among others, which are inherent complexities and the subtleties in their manifestations within the data.

Dropout randomly drops out individual neurons by setting their activation to zero at every training step. This is done by applying a mask to the neurons, effectively turning off a subset of them. This forces the network to not over-rely on specific neurons and adapt to various activation patterns. The outputs of a Dropout layer will then be

$$Dropout(\mathcal{X}'') = (\mathcal{X}'' * \mathcal{W}_{dropout}) \odot \mathcal{M}, \tag{18}$$

$$\mathcal{M}_{ij} \sim \text{Bernoulli}(p), \tag{19}$$

where $\mathcal{M}$ is a mask matrix whose elements $\mathcal{M}_{ij}$ follow a Bernoulli distribution (i.e., each element is 1 with probability p and 0 with probability $1-p$). The Hadamard product $\odot$ denotes element-wise multiplication.

By this method, a random subset of neurons is "dropped out" (i.e., their outputs are set to zero) during training, which helps to prevent overfitting.

DropFusion extends the dropout technique by applying two separate masks to the connections (edges) and neurons at every training step, forcing the network to not over-rely on specific edges or neurons and adapt to various connectivity and activation patterns. The outputs of a DropFusion layer will then be written as

$$DropFusion(\mathcal{X}'') = (\mathcal{X}'' * (\mathcal{W}_{dropfusion} \odot \mathcal{M}1)) \odot \mathcal{M}2, \tag{20}$$

$$\mathcal{M}1_{ij} \sim \text{Bernoulli}(p_1), \mathcal{M}2_{ij} \sim \text{Bernoulli}(p_2), \tag{21}$$

where two mask matrices $\mathcal{M}_1$ and $\mathcal{M}_2$ are introduced, each following different Bernoulli distributions p_1 and p_2, respectively. First, the weight matrix $\mathcal{W}_{dropfusion}$ is masked with $\mathcal{M}_1$ to drop edges, and then the intermediate result is masked again with $\mathcal{M}_2$ to drop neurons.

To ensure consistency between the expected output during training and testing, it is necessary to scale the output. During training, some edges and neurons are dropped, leading to a lower expected output compared to testing. Therefore, to compensate for the dropped connections and neurons, the remaining neurons' outputs must be scaled by an appropriate factor. For the first mask $\mathcal{M}_1$ (edge mask), the scaling factor $\frac{1}{1-p_1}$ is applied. This factor accounts for the Bernoulli distribution of $\mathcal{M}_1$, which determines the probability p_1 of dropping an edge.

$$\mathcal{W}_{dropfusion} \odot \mathcal{M}_1 \rightarrow \frac{1}{1-p_1}(\mathcal{W}_{dropfusion} \odot \mathcal{M}_1) \tag{22}$$

For the second mask $\mathcal{M}_2$ (neuron mask), the output of the neurons is similarly scaled by $\frac{1}{1-p_2}$.

$$\begin{aligned} &\left(\mathcal{X}'' * (\mathcal{W}_{dropfusion} \odot \mathcal{M}_1)\right) \odot \mathcal{M}_2 \\ &\rightarrow \frac{1}{1-p_2}\left(\mathcal{X}'' * (\mathcal{W}_{dropfusion} \odot \mathcal{M}_1)\right) \odot \mathcal{M}_2 \end{aligned} \tag{23}$$

Thus, the output after two mask operations and scaling will be:

$$DropFusion(\mathcal{X}'') = \frac{1}{(1-p_1)(1-p_2)}\left(\mathcal{X}'' * (\mathcal{W}_{dropfusion} \odot \mathcal{M}_1)\right) \odot \mathcal{M}_2 \tag{24}$$

5.4. IntraVBlock: intra-variable dependency learning

Furthermore, based on the fusion of channel correlations, we capture the internal temporal dependencies of the variables. Before inputting into the IntraVBlock, we need to transpose the data, while also incorporating a residual connection mechanism to prevent the loss of crucial information. Additionally, given that the temporal dependencies are more tightly coupled, we use a dense Feed-Forward Network (FFN) for regression learning. As shown in Fig. 7, we employ two fully connected layers and a nonlinear activation function for temporal analysis on each channel, similar to current research on linear forecasters that utilize Multi-Layer Perceptrons (MLPs) to extract temporal features. MLP-based forecasters have been shown to effectively capture the intrinsic properties of time series [2,39]. Consequently, by stacking blocks, the MLP is dedicated to encoding the observed time series and decoding the representations for future series using dense nonlinear connections.

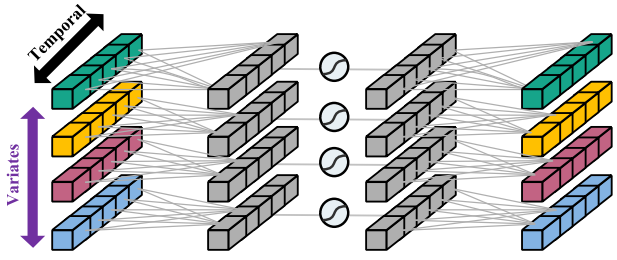


Fig. 7. Intra-Variable block for series representations.

$$\mathcal{X}''' \leftarrow CrossVBlock(\mathcal{X}'')^T + \mathcal{X}' \tag{25}$$

$$IntraVBlock(\mathcal{X}''') = FFN(\mathcal{X}''') \tag{26}$$

6. Experiment

Contrary to some recent Transformer-based advances with increased complexity, DyTimeNet offers a relatively simple architecture. Despite its simplicity, we demonstrate that DyTimeNet remains competitive with state-of-the-art models on representative benchmarks. All the experiments are conducted on a single Tesla V100 GPU with 16GB of memory.

Datasets. We conduct experiments on 13 real-world datasets. For long-term forecasting, we consider the Weather, Illness, Exchange, Electricity, Traffic, and ETT datasets (including 4 subsets) following TimesNet [40]. For short-term forecasting, we adopt the PEMS [41] which contains four public traffic network datasets (PEMS03, PEMS04, PEMS07, PEMS08). More details of the datasets are provided in Table 2.

(1) **ETT** [2]: It contains 7 factors of electricity transformers from July 2016 to July 2018. There are four subsets including two hourly-level datasets (ETTh1 and ETTh2), and two 15-minute-level datasets (ETTm1 and ETTm2).
(2) **Exchange** [3]: It contains the collection of the daily exchange rates of 8 foreign countries including Australia, Britain, Canada, Switzerland, China, Japan, New Zealand, and Singapore ranging from 1990 to 2016 with the data sampling interval of 1 day.
(3) **Weather** [4]: It collects 21 meteorological indicators, such as humidity and air temperature, from the Weather Station of the Max Planck Biogeochemistry Institute in Germany in 2020. The data sampling interval is every 10 minutes.
(4) **Traffic [42]**: It contains hourly road occupancy rates recorded by 862 sensors of San Francisco Bay area freeways from January 2015 to December 2016.
(5) **Electricity** [5]: It records hourly electricity consumption of 321 clients since 01/01/2011.
(6) **Illness(ILI)** [6]: It includes weekly recorded data on the number of patients with influenza-like illnesses in the United States from 2002 to 2021.
(7) **PEMS** [7]: It contains four public traffic network datasets (PEMS03, PEMS04, PEMS07 and PEMS08) which are respectively constructed from Caltrans Performance Measurement System (PeMS) of four districts in California. The data is aggregated into 5-minute windows, resulting in 12 points per hour and 288 points per day.

Experimental Settings. The relevant hyperparameter settings are provided in Table 3. To ensure a fair comparison of performance across different methods, the batch size for all experiments is set to 32. For simplicity, we set the mask probabilities p_1 and p_2 to 0.5 (within the range of 0 to 1). However, it is important to note that the optimal values of p_1 and p_2 may vary across different datasets. Therefore, in Section 6.4, we conduct experiments to explore the optimal combinations of p_1 and p_2 for several datasets.

Baselines. We benchmark DyTimeNet against 18 state-of-the-art forecasting models, including (1) **Transformer-based:** iTransformer [9], Autoformer [43], FEDformer [44], Stationary [45], Crossformer

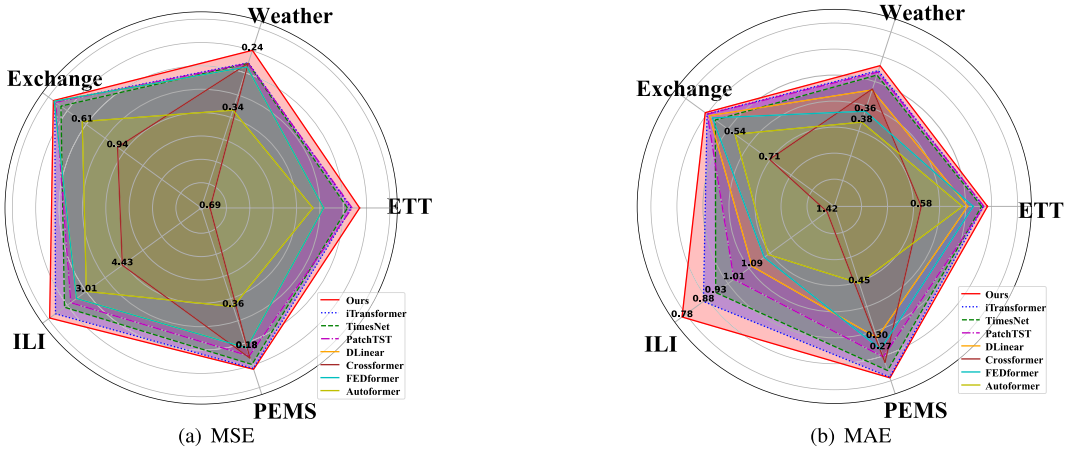
[2] https://github.com/zhouhaoyi/ETDataset.
[3] https://github.com/laiguokun/multivariate-time-series-data.
[4] https://www.bgc-jena.mpg.de/wetter/.
[5] https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014.
[6] https://gis.cdc.gov/grasp/fluview/fluportaldashboard.html.
[7] https://pems.dot.ca.gov/.

Table 2
Summary of experiment benchmarks.

Tasks	Dataset	Variate	Predict Length	Frequency	Domain
Long-term forecasting	ETT (4 subsets)	7	96 ~ 720	15 mins/Hourly	Temperature
	Illness	7	24 ~ 60	Weekly	Illness
	Exchange	8	96 ~ 720	Daily	Economy
	Weather	21	96 ~ 720	10 mins	Weather
	Electricity	321	96 ~ 720	Hourly	Electricity
	Traffic	862	96 ~ 720	Hourly	Transportation
Short-term forecasting	PEMS (4 subsets)	170 ~ 883	12 ~ 24	5 mins	Traffic network

Table 3
Detailed hyperparameter settings for thirteen datasets. "d_model" refers to the hidden dimension of the model.

Datasets	ILI	Electricity	Weather	ETTh1	ETTh2	ETTm1	ETTm2	Traffic	Exchange	PEMS03	PEMS04	PEMS07	PEMS08
# d_model	64	64	32	16	64	32	64	64	32	64	64	64	64
# batch size	32	32	32	32	32	32	32	32	32	32	32	32	32
# train epoch	20	30	20	20	20	20	20	50	10	50	20	50	50
# learning rate	0.0005	0.0005	0.0005	0.0005	0.0005	0.0005	0.0005	0.0005	0.0005	0.0005	0.0005	0.0005	0.0005
# $p1$	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5
# $p2$	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5	0.5



(a) MSE (b) MAE

Fig. 8. Comparison of model performance.

[11], PatchTST [1]; (2) **Linear-based:** TimeMixer [5], DLinear [4], TSMixer [23], TiDE [39], LightTS [20]; (3) **CNN-based:** TimesNet [40]; (4) **LLM-based:** GPT4TS [16], UniTime [17], Moment [46], S-Mamba [47] and MOIRAI [48]; and (5) **Dual-level models:** CrossGNN [28], DSIN-PMA [15].

Evaluation Metric. Following previous works [5,9], we use Mean Squared Error (MSE) and Mean Absolute Error (MAE) as the core metrics to compare performance. A lower MSE or MAE indicates more accurate forecasting results. These metrics can be calculated as follows:

$$MSE = \frac{1}{S}\sum_{i=1}^{S}(y_i - \hat{y}_i)^2 \tag{27}$$

$$MAE = \frac{1}{S}\sum_{i=1}^{S}|y_i - \hat{y}_i| \tag{28}$$

6.1. Main results

Experimentally, as shown in Fig. 8, DyTimeNet consistently demonstrates leading performance on real-world forecasting benchmarks, surpassing other customized models in both MSE and MAE.

Long-term forecasting. As shown in Table 4, DyTimeNet shows great performance in most benchmarks (38/45 scenarios), outperforming the advanced iTransformer [9]. This superiority is attributed to the ability of CrossVBlock to capture the multiple correlations among variates, unlike the linear relationships learned by the Attention mechanism of iTransformer. Moreover, compared with DSIN-PMA [15], whose dual-path framework is built upon Pearson-based linear channel relationships, DyTimeNet achieves better results by capturing more complex nonlinear interactions among channels. Additionally, it is worth noting that DyTimeNet outperforms the latest TimeMixer [5], as well as other Linear-based methods (such as DLinear [4], LightTS [20], and TSMixer [23]) by a significant margin. This is because these methods consider channels to be independent, overlooking the information gain brought by channels and thus only capturing internal temporal dependencies, further proving the effectiveness of DyTimeNet. Finally, no single model excels across all datasets, as different methods are better suited for different types of multivariate time series. For example, DSIN-PMA and iTransformer excel in purely linear synchronous time series, while PatchTST performs better for datasets with independent channels. In contrast, DyTimeNet's sparse strategy makes it capable of modeling a mix of linear, nonlinear, and independent relationships, allowing it to handle a wider range of scenarios.

Table 4
Long-term forecasting full results. The input sequence length is set to 36 for the Illness dataset and 96 for the others. The predictive lengths are set to {24, 36, 48, 60} for Illness, and {96, 192, 336, 720} for others. AVG is averaged over all predictive lengths. A lower MSE or MAE indicates a better prediction, with the best results in **bold**. We fix the input length as 36 for ILI and 96 for the others.

Models		DyTimeNet		DSIN-PMA		CrossGNN		TimeMixer		iTransformer		TimesNet		PatchTST		DLinear		TSMixer		LightTS		Crossformer		FEDformer		Autoformer	
dataset	pred_len	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	**0.310**	**0.352**	0.312	0.352	0.336	0.375	0.320	0.357	0.334	0.368	0.338	0.375	0.329	0.367	0.345	0.372	0.533	0.533	0.374	0.400	0.404	0.426	0.379	0.419	0.505	0.475
	192	**0.354**	**0.378**	0.357	0.375	0.373	0.392	0.361	0.381	0.377	0.391	0.374	0.387	0.367	0.385	0.380	0.389	0.425	0.425	0.400	0.407	0.450	0.451	0.426	0.441	0.553	0.496
	336	**0.387**	**0.401**	0.398	0.402	0.411	0.415	0.390	0.404	0.426	0.420	0.410	0.411	0.399	0.410	0.413	0.413	0.466	0.461	0.438	0.438	0.532	0.515	0.445	0.459	0.621	0.537
	720	**0.447**	**0.436**	0.469	0.442	0.461	0.442	0.454	0.441	0.491	0.459	0.478	0.450	0.454	0.439	0.474	0.453	0.530	0.503	0.527	0.502	0.666	0.589	0.543	0.490	0.671	0.561
	AVG	**0.374**	**0.392**	0.384	0.392	0.395	0.406	0.381	0.395	0.407	0.410	0.400	0.406	0.387	0.400	0.403	0.407	0.488	0.481	0.435	0.437	0.513	0.496	0.448	0.452	0.588	0.517
ETTm2	96	**0.172**	**0.254**	0.173	0.253	0.177	0.265	0.175	0.258	0.180	0.264	0.187	0.267	0.175	0.259	0.193	0.292	0.475	0.515	0.209	0.308	0.287	0.366	0.203	0.287	0.255	0.339
	192	**0.234**	**0.296**	0.236	0.297	0.242	0.307	0.237	0.299	0.250	0.309	0.249	0.309	0.241	0.302	0.284	0.362	0.515	0.385	0.311	0.382	0.414	0.492	0.269	0.328	0.281	0.340
	336	**0.296**	**0.335**	0.303	0.339	0.306	0.348	0.298	0.340	0.311	0.348	0.321	0.351	0.305	0.343	0.369	0.427	0.735	0.658	0.442	0.466	0.597	0.542	0.325	0.366	0.339	0.372
	720	**0.395**	**0.395**	0.399	0.396	0.406	0.402	0.391	0.396	0.412	0.407	0.408	0.403	0.402	0.400	0.554	0.522	1.035	0.771	0.675	0.587	1.730	1.042	0.421	0.415	0.433	0.432
	AVG	**0.274**	**0.320**	0.278	0.321	0.283	0.331	0.275	0.323	0.288	0.332	0.291	0.333	0.281	0.326	0.350	0.401	0.690	0.582	0.409	0.436	0.757	0.610	0.305	0.349	0.327	0.371
ETThl	96	**0.371**	**0.393**	0.369	0.389	0.381	0.371	0.375	0.400	0.386	0.405	0.384	0.402	0.414	0.419	0.386	0.400	0.722	0.600	0.424	0.432	0.423	0.448	0.376	0.419	0.449	0.459
	192	**0.428**	**0.427**	0.418	0.419	0.429	0.426	0.429	0.421	0.441	0.436	0.436	0.429	0.460	0.445	0.437	0.432	0.602	0.533	0.475	0.462	0.471	0.474	0.420	0.448	0.500	0.482
	336	**0.472**	**0.448**	0.462	0.442	0.467	0.447	0.484	0.458	0.487	0.458	0.491	0.469	0.501	0.466	0.481	0.459	0.650	0.565	0.518	0.488	0.570	0.546	0.459	0.465	0.521	0.496
	720	**0.473**	**0.465**	0.476	0.462	0.472	0.468	0.498	0.482	0.503	0.491	0.521	0.500	0.500	0.488	0.519	0.516	0.627	0.582	0.547	0.533	0.653	0.621	0.506	0.507	0.514	0.512
	AVG	**0.436**	**0.433**	0.431	0.428	0.437	0.428	0.447	0.440	0.454	0.448	0.458	0.450	0.469	0.455	0.456	0.452	0.650	0.570	0.491	0.479	0.529	0.522	0.440	0.460	0.496	0.487
ETTh2	96	0.294	0.345	**0.288**	**0.335**	0.311	0.361	0.289	0.341	0.297	0.349	0.340	0.372	0.302	0.348	0.333	0.387	0.759	0.657	0.397	0.437	0.745	0.584	0.358	0.397	0.346	0.388
	192	0.374	0.394	**0.362**	**0.384**	0.392	0.408	0.372	0.392	0.380	0.400	0.402	0.414	0.388	0.400	0.477	0.476	0.552	0.550	0.520	0.504	0.877	0.656	0.429	0.439	0.456	0.452
	336	0.412	**0.426**	**0.410**	0.426	0.426	0.443	0.386	0.414	0.428	0.432	0.452	0.452	0.426	0.433	0.594	0.541	0.693	0.593	0.626	0.559	1.043	0.731	0.496	0.487	0.482	0.486
	720	**0.424**	**0.443**	0.429	0.445	0.446	0.466	0.412	0.434	0.427	0.445	0.462	0.468	0.431	0.446	0.831	0.657	0.877	0.698	0.863	0.672	1.104	0.763	0.463	0.474	0.515	0.511
	AVG	0.376	0.402	**0.372**	**0.398**	0.394	0.420	0.364	0.395	0.383	0.407	0.414	0.427	0.387	0.407	0.559	0.515	0.720	0.624	0.602	0.543	0.942	0.684	0.437	0.449	0.450	0.459
Weather	96	**0.155**	**0.202**	0.169	0.209	0.161	0.218	0.163	0.209	0.174	0.214	0.172	0.220	0.177	0.218	0.196	0.255	0.177	0.251	0.182	0.242	0.158	0.230	0.217	0.296	0.266	0.336
	192	**0.203**	**0.245**	0.212	0.249	0.213	0.268	0.208	0.250	0.221	0.254	0.219	0.261	0.225	0.259	0.237	0.296	0.219	0.286	0.227	0.287	0.206	0.277	0.276	0.336	0.307	0.367
	336	**0.250**	**0.286**	0.267	0.290	0.268	0.315	0.251	0.287	0.278	0.296	0.280	0.306	0.278	0.297	0.283	0.335	0.269	0.319	0.282	0.334	0.272	0.335	0.339	0.380	0.359	0.395
	720	**0.338**	**0.340**	0.347	0.342	0.356	0.369	0.339	0.341	0.358	0.349	0.365	0.359	0.354	0.348	0.345	0.381	0.319	0.359	0.352	0.386	0.398	0.418	0.403	0.428	0.419	0.428
	AVG	**0.237**	**0.268**	0.249	0.273	0.250	0.293	0.240	0.271	0.258	0.278	0.259	0.287	0.259	0.281	0.265	0.317	0.246	0.304	0.261	0.312	0.259	0.315	0.309	0.360	0.338	0.382
Exchange	96	**0.080**	**0.198**	0.086	0.204	0.086	0.203	0.084	0.084	0.086	0.206	0.107	0.234	0.088	0.205	0.088	0.218	0.246	0.246	0.116	0.262	0.256	0.367	0.148	0.278	0.197	0.323
	192	**0.171**	**0.293**	0.178	0.299	0.173	0.296	0.188	0.308	0.177	0.299	0.226	0.344	0.176	0.299	0.176	0.315	0.467	0.550	0.215	0.359	0.470	0.509	0.271	0.315	0.300	0.369
	336	**0.313**	**0.403**	0.330	0.415	0.321	0.410	0.353	0.428	0.331	0.417	0.367	0.448	0.301	0.397	0.313	0.427	0.533	0.583	0.377	0.466	1.268	0.883	0.460	0.427	0.509	0.524
	720	**0.821**	**0.680**	0.839	0.690	0.806	0.678	0.891	0.709	0.847	0.691	0.964	0.746	0.901	0.714	0.839	0.695	0.726	0.714	0.831	0.699	1.767	1.068	1.195	0.695	1.447	0.941
	AVG	**0.346**	**0.394**	0.358	0.402	0.347	0.397	0.379	0.382	0.360	0.403	0.416	0.443	0.367	0.404	0.354	0.414	0.493	0.523	0.385	0.447	0.940	0.707	0.519	0.429	0.613	0.539
ILI	24	**1.522**	**0.771**	2.480	1.052	2.564	1.517	2.183	0.908	1.905	0.856	2.317	0.934	2.335	0.989	2.398	1.040	6.296	1.803	8.313	2.144	4.313	1.428	3.228	1.260	3.483	1.287
	36	**1.383**	**0.726**	2.315	1.010	2.626	1.496	1.813	0.808	1.590	0.915	1.972	0.920	2.561	1.035	2.646	1.088	6.843	1.889	6.631	1.902	4.195	1.361	2.679	1.080	3.103	1.148
	48	**1.482**	**0.776**	2.338	1.025	2.733	1.602	2.048	0.853	1.654	0.825	2.238	0.940	2.465	1.022	2.614	1.086	6.729	1.865	7.299	1.982	4.545	1.428	2.622	1.078	2.669	1.085
	60	**1.812**	**0.859**	2.251	1.004	2.817	1.856	2.133	2.133	1.974	0.915	2.027	0.928	2.189	0.997	2.804	1.146	6.866	1.888	7.283	1.985	4.681	1.464	2.857	1.157	2.770	1.125
	AVG	**1.550**	**0.783**	2.346	1.023	2.685	1.618	2.044	1.176	1.781	0.877	2.139	0.931	2.388	1.011	2.616	1.090	6.683	1.861	7.382	2.003	4.434	1.420	2.847	1.144	3.006	1.161
Traffic	96	**0.355**	**0.208**	0.420	0.277	0.577	0.315	0.462	0.285	0.395	0.268	0.593	0.321	0.544	0.359	0.650	0.396	0.626	0.411	0.615	0.391	0.522	0.290	0.587	0.366	0.613	0.388
	192	**0.362**	**0.211**	0.440	0.282	0.578	0.323	0.473	0.296	0.417	0.276	0.617	0.336	0.540	0.354	0.598	0.370	0.637	0.422	0.601	0.382	0.530	0.293	0.604	0.373	0.616	0.382
	336	**0.381**	**0.219**	0.455	0.288	0.587	0.322	0.498	0.296	0.433	0.283	0.629	0.336	0.551	0.358	0.605	0.373	0.642	0.427	0.613	0.386	0.558	0.305	0.621	0.383	0.622	0.337
	720	**0.416**	**0.239**	0.483	0.304	0.597	0.335	0.506	0.313	0.467	0.302	0.640	0.350	0.586	0.375	0.645	0.394	0.738	0.468	0.658	0.407	0.589	0.328	0.626	0.382	0.660	0.408
	AVG	**0.379**	**0.219**	0.450	0.288	0.585	0.324	0.484	0.297	0.428	0.282	0.620	0.336	0.555	0.362	0.625	0.383	0.661	0.432	0.622	0.392	0.550	0.304	0.610	0.376	0.628	0.379
Electricity	96	**0.147**	**0.239**	0.155	0.246	0.175	0.276	0.153	0.247	0.148	0.240	0.168	0.272	0.195	0.285	0.197	0.282	0.224	0.321	0.207	0.307	0.219	0.314	0.193	0.308	0.201	0.317
	192	**0.155**	**0.252**	0.170	0.258	0.196	0.291	0.166	0.256	0.162	0.253	0.184	0.289	0.199	0.289	0.196	0.285	0.242	0.341	0.213	0.316	0.231	0.322	0.201	0.315	0.222	0.334
	336	**0.172**	**0.263**	0.185	0.274	0.206	0.311	0.185	0.277	0.178	0.269	0.198	0.300	0.215	0.305	0.209	0.301	0.239	0.343	0.230	0.333	0.246	0.337	0.214	0.329	0.231	0.338
	720	**0.212**	**0.304**	0.222	0.303	0.233	0.336	0.225	0.310	0.225	0.317	0.220	0.320	0.256	0.337	0.245	0.333	0.276	0.373	0.265	0.360	0.280	0.363	0.246	0.355	0.254	0.361
	AVG	**0.172**	**0.265**	0.183	0.270	0.203	0.304	0.182	0.272	0.178	0.270	0.192	0.295	0.216	0.304	0.212	0.300	0.246	0.344	0.229	0.329	0.244	0.334	0.214	0.327	0.227	0.338

For fairness, all experimental results use the same input length and batch size.

Table 5
Short-term forecasting results on the PEMS datasets with multiple variates. All input lengths are 96 and prediction lengths are {12,24}. A lower MSE and MAE indicate a better prediction, with the best results shown in **bold**.

Models		DyTimeNet		DSIN-PMA		CrossGNN		iTransformer		TimesNet		PatchTST		DLinear		TiDE		Crossformer		FEDformer		Stationary	
dataset	pred_len	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
PEMS03	12	**0.066**	**0.173**	0.076	0.184	0.084	0.201	0.071	0.174	0.085	0.192	0.099	0.216	0.122	0.243	0.178	0.305	0.090	0.203	0.126	0.251	0.081	0.188
	24	**0.090**	**0.198**	0.110	0.223	0.137	0.253	0.093	0.201	0.118	0.223	0.142	0.259	0.201	0.317	0.257	0.371	0.121	0.240	0.149	0.275	0.105	0.214
	AVG	**0.078**	**0.186**	0.093	0.203	0.111	0.227	0.082	0.188	0.102	0.208	0.121	0.238	0.162	0.280	0.218	0.338	0.106	0.222	0.138	0.263	0.093	0.201
PEMS04	12	**0.075**	**0.183**	0.093	0.200	0.098	0.221	0.078	0.183	0.087	0.195	0.105	0.224	0.148	0.272	0.219	0.340	0.098	0.218	0.138	0.262	0.088	0.196
	24	**0.091**	**0.203**	0.130	0.240	0.146	0.263	0.095	0.205	0.103	0.215	0.153	0.275	0.224	0.340	0.292	0.398	0.131	0.256	0.177	0.293	0.104	0.216
	AVG	**0.083**	**0.193**	0.111	0.220	0.122	0.242	0.087	0.194	0.095	0.205	0.129	0.250	0.186	0.306	0.256	0.369	0.115	0.237	0.158	0.278	0.096	0.206
PEMS07	12	**0.061**	**0.162**	0.066	0.168	0.078	0.189	0.067	0.165	0.082	0.181	0.095	0.207	0.115	0.242	0.173	0.304	0.094	0.200	0.109	0.225	0.083	0.185
	24	**0.081**	**0.186**	0.098	0.205	0.115	0.245	0.088	0.190	0.101	0.204	0.150	0.262	0.210	0.329	0.271	0.383	0.139	0.247	0.125	0.244	0.102	0.207
	AVG	**0.071**	**0.174**	0.082	0.186	0.097	0.217	0.078	0.178	0.092	0.193	0.123	0.235	0.163	0.286	0.222	0.344	0.117	0.224	0.117	0.235	0.093	0.196
PEMS08	12	**0.077**	**0.181**	0.088	0.192	0.103	0.204	0.079	0.182	0.112	0.212	0.168	0.232	0.154	0.276	0.227	0.343	0.165	0.214	0.173	0.273	0.109	0.207
	24	**0.114**	**0.218**	0.136	0.239	0.163	0.261	0.115	0.219	0.141	0.238	0.224	0.281	0.248	0.353	0.318	0.409	0.215	0.260	0.210	0.301	0.140	0.236
	AVG	**0.096**	**0.200**	0.112	0.216	0.133	0.233	0.097	0.201	0.127	0.225	0.196	0.257	0.201	0.315	0.273	0.376	0.190	0.237	0.192	0.287	0.125	0.222
PEMS AVG		**0.082**	**0.188**	0.100	0.206	0.116	0.230	0.086	0.190	0.104	0.208	0.142	0.245	0.178	0.297	0.242	0.357	0.132	0.230	0.151	0.266	0.102	0.206

Table 6
The full comparison results of LLM-based methods with different lookback sequences {96, 512}. The predictive lengths are set to {24, 36, 48, 60} for Illness, and {96, 192, 336, 720} for others. AVG is averaged over all predictive lengths. A lower MSE or MAE indicates a better prediction, with the best results shown in **bold**.

lookback_seq = 96		DyTimeNet		UniTime*		GPT4TS*		S-Mamba*		lookback_seq = 512		DyTimeNet		GPT4TS*		Moment*		MOIRAI*	
Dataset	pred_len	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	Dataset	pred_len	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	**0.310**	**0.352**	0.322	0.363	0.331	0.364	0.329	0.364	ETTm1	96	**0.283**	**0.341**	0.292	0.346	0.293	0.349	0.353	0.363
	192	**0.354**	**0.378**	0.366	0.387	0.371	0.384	0.371	0.387		192	**0.325**	**0.366**	0.332	0.372	–	–	0.376	0.380
	336	**0.387**	**0.401**	0.398	0.407	0.401	0.405	0.403	0.409		336	**0.356**	**0.390**	0.366	0.394	–	–	0.399	0.395
	720	**0.447**	**0.436**	0.454	0.440	0.463	0.440	0.471	0.448		720	**0.402**	**0.412**	0.417	0.421	0.405	0.416	0.432	0.417
	AVG	**0.374**	**0.392**	0.385	0.399	0.392	0.398	0.394	0.402		AVG	**0.341**	**0.377**	0.352	0.383	0.349	0.383	0.390	0.389
ETTm2	96	**0.172**	**0.254**	0.183	0.266	0.177	0.263	0.179	0.262	ETTm2	96	**0.169**	**0.260**	0.173	0.262	0.170	0.260	0.189	0.260
	192	**0.234**	**0.296**	0.251	0.310	0.245	0.306	0.245	0.307		192	**0.226**	**0.298**	0.229	0.301	–	–	0.247	0.300
	336	**0.296**	**0.335**	0.319	0.351	0.310	0.348	0.309	0.347		336	**0.278**	**0.331**	0.286	0.341	–	–	0.295	0.334
	720	**0.395**	**0.395**	0.420	0.410	0.408	0.407	0.406	0.402		720	**0.362**	**0.387**	0.378	0.401	0.363	0.387	0.372	0.386
	AVG	**0.274**	**0.320**	0.293	0.334	0.285	0.331	0.285	0.330		AVG	**0.259**	**0.319**	0.266	0.326	0.267	0.324 0.276	0.320	
ETTh1	96	**0.371**	**0.394**	0.397	0.418	0.376	0.397	0.385	0.405	ETTh1	96	**0.367**	**0.396**	0.376	0.397	–	–	0.380	0.398
	192	**0.428**	**0.427**	0.434	0.439	0.430	0.428	0.433	0.433		192	**0.406**	**0.423**	0.416	0.418	–	–	0.440	0.434
	336	0.472	**0.448**	**0.468**	0.457	0.474	0.449	0.475	0.455		336	**0.434**	**0.444**	0.442	0.433	–	–	0.514	0.474
	720	0.473	**0.465**	**0.469**	0.477	0.478	0.468	0.482	0.479		720	**0.470**	**0.479**	0.477	0.456	–	–	0.705	0.568
	AVG	**0.436**	**0.433**	0.442	0.448	0.440	0.435	0.444	0.443		AVG	**0.419**	**0.435**	0.427	0.426	–	–	0.510	0.469
ETTh2	96	**0.294**	**0.345**	0.296	0.345	0.299	0.351	0.294	0.348	ETTh2	96	**0.285**	**0.340**	0.285	0.342	0.288	0.345	0.287	0.325
	192	**0.374**	**0.394**	0.374	0.394	0.378	0.399	0.376	0.398		192	**0.338**	**0.380**	0.354	0.389	–	–	0.347	0.367
	336	**0.412**	**0.426**	0.415	0.427	0.428	0.437	0.417	0.430		336	**0.360**	**0.403**	0.373	0.407	–	–	0.377	0.393
	720	**0.424**	**0.443**	0.425	0.444	0.431	0.451	0.434	0.449		720	**0.401**	**0.436**	0.406	0.441	0.403	0.439	0.404	0.421
	AVG	**0.376**	**0.402**	0.378	0.403	0.384	0.410	0.380	0.406		AVG	**0.346**	**0.390**	0.354	0.394	0.346	0.392	0.354	0.377
Weather	96	**0.155**	**0.202**	0.171	0.214	0.185	0.225	0.170	0.212	Weather	96	**0.140**	**0.191**	0.162	0.212	0.154	0.209	0.177	0.208
	192	**0.203**	**0.245**	0.217	0.254	0.232	0.252	0.220	0.257		192	**0.183**	**0.234**	0.204	0.248	–	–	0.219	0.249
	336	**0.250**	**0.286**	0.274	0.293	0.287	0.303	0.276	0.298		336	**0.235**	**0.276**	0.254	0.286	–	–	0.277	0.292
	720	**0.338**	**0.340**	0.351	0.343	0.363	0.351	0.356	0.348		720	**0.313**	**0.332**	0.326	0.337	0.315	0.336	0.365	0.350
	AVG	**0.237**	**0.268**	0.253	0.276	0.267	0.283	0.256	0.279		AVG	**0.218**	**0.258**	0.237	0.270	0.235	0.273	0.260	0.275
Electricity	96	0.147	0.239	0.196	0.287	0.197	0.290	**0.139**	**0.235**	Electricity	96	**0.137**	**0.237**	0.139	0.238	0.138	0.242	0.152	0.242
	192	0.165	0.252	0.199	0.291	0.201	0.292	**0.159**	**0.255**		192	**0.152**	**0.250**	0.153	0.251	–	–	0.171	0.259
	336	0.182	0.273	0.214	0.305	0.217	0.309	**0.176**	**0.272**		336	**0.167**	**0.271**	0.169	0.266	–	–	0.192	0.278
	720	0.212	0.304	0.254	0.335	0.253	0.339	**0.204**	**0.298**		720	**0.198**	**0.294**	0.206	0.297	0.211	0.305	0.236	0.313
	AVG	0.177	0.267	0.216	0.305	0.217	0.308	**0.170**	**0.265**		AVG	**0.164**	**0.263**	0.167	0.263	0.175	0.274	0.188	0.273
ILI	24	**1.522**	**0.771**	2.460	0.954	2.073	0.885	–	–	ILI	24	2.239	1.027	**2.063**	**0.881**	2.728	1.114	–	–
	36	**1.383**	**0.726**	1.998	0.912	2.396	0.941	–	–		36	2.276	1.013	**1.868**	**0.892**	–	–	–	–
	48	**1.482**	**0.776**	1.979	0.912	2.234	0.932	–	–		48	2.276	1.023	**1.790**	**0.884**	–	–	–	–
	60	**1.812**	**0.859**	2.109	0.938	2.131	0.944	–	–		60	2.382	1.058	**1.979**	**0.957**	2.893	1.132	–	–
	AVG	**1.550**	**0.783**	2.137	0.929	2.209	0.925	–	–		AVG	2.293	1.030	**1.925**	**0.903**	2.811	1.123	–	–
Exchange	96	**0.080**	**0.198**	0.086	0.209	0.091	0.212	0.086	0.207	Exchange	96	**0.091**	**0.214**	0.100	0.226	–	–	–	–
	192	**0.171**	**0.293**	0.174	0.299	0.183	0.304	0.182	0.304		192	**0.189**	**0.315**	0.218	0.336	–	–	–	–
	336	**0.313**	**0.403**	0.319	0.408	0.328	0.417	0.332	0.418		336	**0.358**	**0.437**	0.421	0.477	–	–	–	–
	720	**0.821**	**0.680**	0.875	0.701	0.880	0.704	0.867	0.703		720	**0.937**	**0.732**	0.991	0.747	–	–	–	–
	AVG	**0.346**	**0.394**	0.364	0.404	0.371	0.409	0.367	0.408		AVG	**0.394**	**0.424**	0.432	0.447	–	–	–	–

* indicates that the experimental results are sourced from the original paper. Therefore, some results may be missing where they were not provided in the original publications.

Short-term forecasting. As shown in Table 5, DyTimeNet also demonstrates excellent performance in short-term forecasting. For the PEMS benchmarks, which record multiple time series from citywide traffic networks, many advanced models, such as PatchTST [1] and DLinear [4], struggle to perform well. This poor performance is due to the complex spatio-temporal correlations among multiple sensors (variates), which are not effectively handled by their channel independence (CI) designs. In contrast, DyTimeNet still performs favorably in this challenging scenario, verifying its effectiveness in handling complex multivariate time series forecasting and capturing localized temporal information.

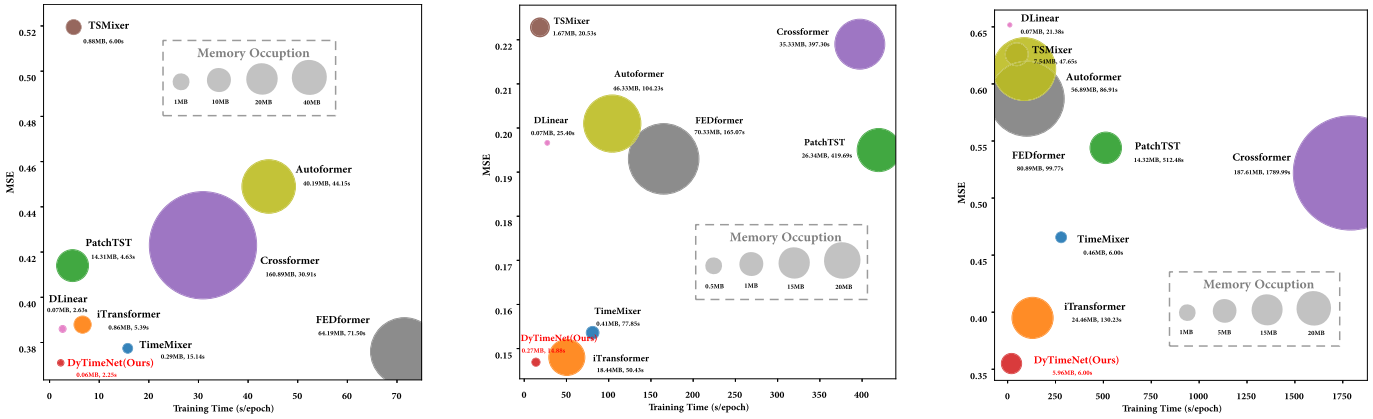
6.2. Comparison with LLM-based methods

To further explore whether DyTimeNet offers a new insight (i.e., rethinking Attention mechanisms for time series) for future large-scale time series models, we compared DyTimeNet with the current LLM-based and Mamba-based [49] methods, including GPT4TS [16], UniTime [17], Moment [46] and S-Mamba [47]. As shown in Table 6, DyTimeNet outperforms existing LLM-based models on most benchmarks. Although LLMs are renowned for their ability to handle long sequences and are theoretically expected to perform well in such scenarios, we extended the input sequence length to 512 for comparison. Surprisingly, DyTimeNet outperformed the LLMs in both long and short sequence tasks. We attribute this to three primary factors: **(1)** The inherent differences between time series and text (as discussed in Introduction), where time series data naturally contain more redundant information due to the higher sampling frequency, leading to increased noise with longer sequences. This makes it difficult for Transformer-based LLMs to leverage their advantages. **(2)** Our series-level variate embedding, through down-sampling strategy, better extracts key temporal information while mitigating redundancy compared to the point-token or patch-token approaches of these LLM-based methods. **(3)** LLM-based methods mentioned above generally assume channel independence, which results in a loss of channel-assisted information. Consequently, even with highly advanced sequence learning capabilities, LLM-based models struggle to compensate for the additional information gains from other channels.

6.3. Model parameters and training speed

We comprehensively evaluate the model's efficiency from three critical perspectives: forecasting performance (MSE), parameter footprint, and training speed. As shown in Fig. 9, we tested the model's efficiency on datasets from three different domains (ETTh1, Electricity and Traffic), and compared DyTimeNet with Transformer-based (e.g., iTransformer, PatchTST, Crossformer) and MLP-based methods (e.g., TimeMixer, TSMixer, DLinear). The results highlight DyTimeNet's advantages in terms of model parameters and inference speed. For instance, as shown the result in Table 7, on the Traffic dataset, DyTimeNet achieves significant improvements compared to Transformer-based methods, such as iTransformer, with **only 1/4** of the parameter size and **7× faster** training. At the same time, DyTimeNet improves the MSE by **10.13%** (from 0.395 to 0.355). Additionally, compared to other MLP-based models such as DLinear, TimeMixer, and TSMixer, DyTimeNet demonstrates higher efficiency in training and inference. This is primarily due to its ability to avoid complex preprocessing steps (such as decomposition) and its effective learning of sparse channel relationships, leading to better predictive performance.

Specifically, on the Electricity dataset, DyTimeNet outperforms the Crossformer model in terms of MSE, with an improvement of **32.88%** (from 0.219 to 0.147). Notably, DyTimeNet achieves this with **only 0.764%** of the training parameters required by Crossformer, while delivering a **26.71× faster** training efficiency. The advantages of DyTimeNet are further confirmed on the Traffic dataset, where it reduces MSE by **31.99%** compared to Crossformer (from 0.522 to 0.355), again with significantly fewer parameters and reduced training time. For the ETTh1 dataset, DyTimeNet also achieves dramatic gains in training efficiency,

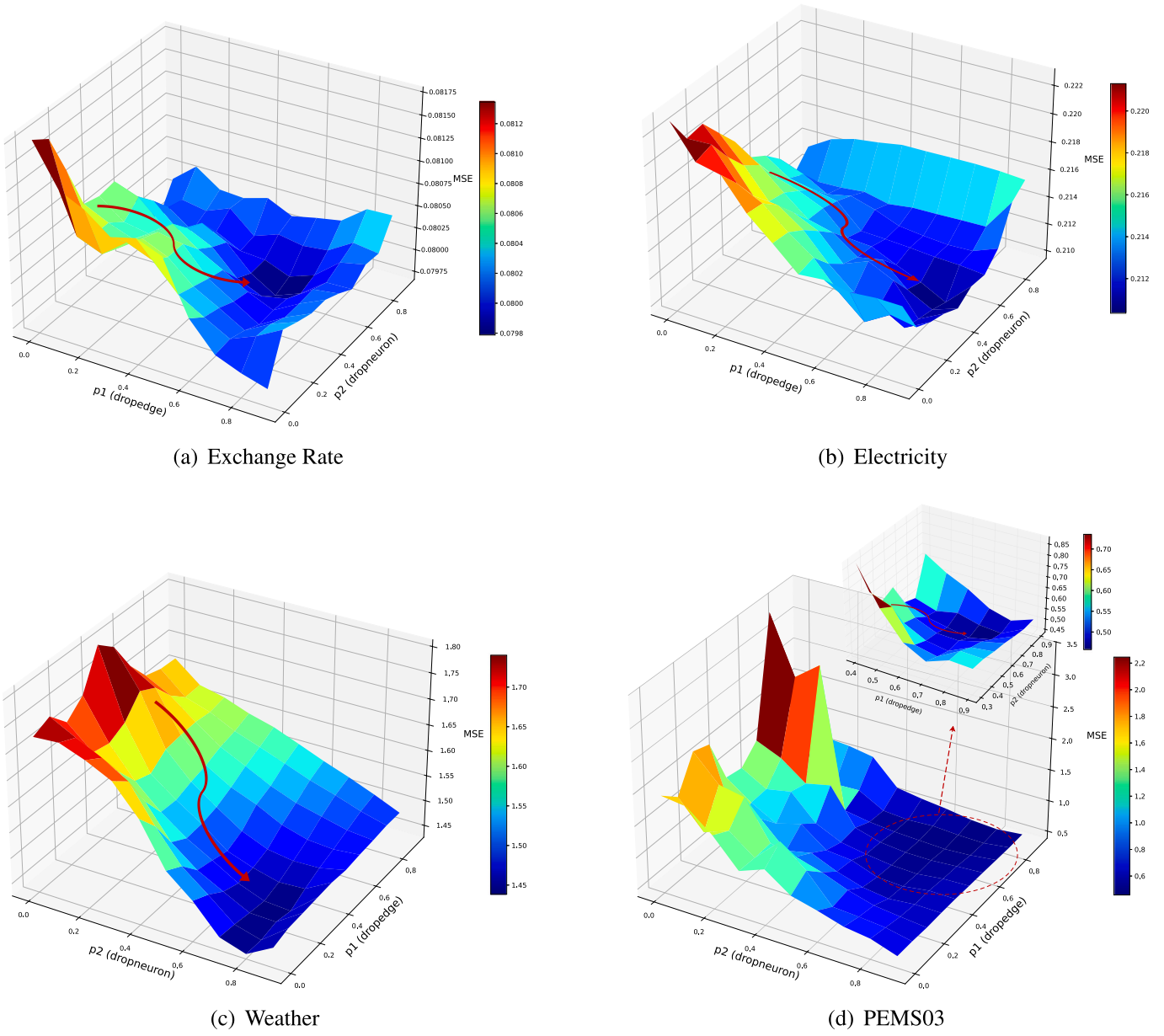


(a) ETTh1 (b) Electricity (c) Traffic

Fig. 9. Model parameters and training Speed.

Table 7
Comparison of model efficiency on datasets ETTh1, Electricity and Traffic during inference phases with Input-96-Prediction-96.

Dataset		DyTimeNet	TimeMixer	iTransformer	PatchTST	Crossformer	TSMixer	DLinear	FEDformer	Autoformer
ETTh1	**memory (MB)**	0.06	0.29	0.86	14.31	160.89	0.88	0.07	64.19	40.19
	time (s)	**2.25**	15.14	5.39	4.63	30.91	6.00	2.63	71.50	44.15
Electricity	**memory (MB)**	0.27	0.41	18.44	26.34	35.33	1.67	0.07	70.33	46.33
	time (s)	**14.88**	77.85	50.43	419.69	397.30	20.53	25.40	165.07	104.23
Traffic	**memory (MB)**	5.96	0.46	24.46	14.32	187.61	7.54	0.07	80.89	56.89
	time (s)	**20.22**	258.02	130.23	512.48	1789.99	47.65	21.38	99.77	86.91



(a) Exchange Rate (b) Electricity

(c) Weather (d) PEMS03

Fig. 10. Visualization of the optimal drop rate search for different datasets, where the x-axis and y-axis represent the drop edge rate (p_1) and drop neuron rate (p_2), respectively, and the z-axis shows the corresponding MSE. The plot illustrates the impact of varying drop rates on model performance.

running **13.73× faster** than Crossformer, while still maintaining competitive forecasting accuracy. These results clearly demonstrate that DyTimeNet not only achieves substantial improvements in predictive performance but also delivers significant gains in efficiency. This combination makes it particularly well-suited for real-world applications where computational resources are constrained and high-frequency, real-time predictions are required.

6.4. Hyperparameter tuning

For consistency with the experiments reported above, we set the drop-edge rate p_1 and the drop-neuron rate p_2 to 0.5 by default. Nevertheless, the optimal (p_1, p_2) can vary across datasets because cross-variable dependency patterns differ substantially among application scenarios. In general, when inter-variable relationships are stronger and denser, smaller drop rates are preferred to avoid masking informative dependencies. When relationships are sparser or noisier, larger drop rates are often beneficial because they suppress irrelevant connections and reduce interference. In this subsection, we examine the sensitivity of DropFusion to (p_1, p_2) using four representative scenarios, including finance (Exchange Rate), electricity load (ECL), weather (Weather), and traffic (PEMS03). For efficiency, we randomly sample 1500 instances from each dataset for this analysis. As shown in Fig. 10, each dataset exhibits a distinct optimal region. Specifically, Exchange Rate achieves the lowest MSE around $p_1 = 0.6$ and $p_2 = 0.6$, ECL performs best around $p_1 = 0.8$ and $p_2 = 0.8$, Weather around $p_1 = 0.3$ and $p_2 = 0.7$, and PEMS03 around $p_1 = 0.7$ and $p_2 = 0.7$. These observations are consistent with the correlation characteristics of the datasets. For example, as illustrated in Fig. 18(b), Weather contains 21 variables with relatively sparse cross-variable connections. This motivates a higher p_2 to encourage robustness at the neuron level, while a lower p_1 helps preserve the limited but informative inter-variable edges. In contrast, ECL and PEMS03 contain many variables and tend to exhibit weaker inter-variable correlations. Therefore, larger values of both p_1 and p_2 are more effective in filtering noisy connections and reducing cross-variable interference.

Table 8
Ablation study on cross-variable block and intra-variable block.

Model		DyTimeNet		w/o_CrossVBlock		w/o_IntraVBlock		IntraVBlock → CrossVBlock	
dataset	pred_len	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	**0.310**	**0.352**	0.320	0.359	0.332	0.366	0.312	0.353
	192	**0.354**	**0.378**	0.365	0.384	0.371	0.385	0.357	0.379
	336	**0.387**	**0.401**	0.397	0.406	0.405	0.406	0.386	0.402
	720	**0.447**	**0.436**	0.459	0.444	0.471	0.443	0.448	0.437
	AVG	**0.374**	**0.392**	0.385	0.398	0.395	0.400	0.375	0.393
ETTm2	96	**0.172**	**0.254**	0.177	0.259	0.174	0.258	0.173	0.256
	192	**0.234**	**0.296**	0.241	0.301	0.244	0.302	0.240	0.299
	336	**0.296**	**0.335**	0.302	0.342	0.304	0.339	0.302	0.342
	720	**0.395**	**0.395**	0.396	0.396	0.402	0.396	0.396	0.396
	AVG	**0.274**	**0.320**	0.279	0.324	0.281	0.324	0.278	0.323
Weather	96	**0.155**	**0.202**	0.178	0.218	0.164	0.208	0.156	0.203
	192	**0.203**	**0.245**	0.224	0.257	0.210	0.249	0.203	0.246
	336	**0.250**	**0.286**	0.281	0.298	0.267	0.291	0.260	0.289
	720	**0.338**	**0.340**	0.356	0.347	0.349	0.344	0.339	0.341
	AVG	**0.237**	**0.268**	0.260	0.280	0.248	0.273	0.239	0.269
ILI	24	**1.522**	**0.771**	1.765	0.869	1.981	0.877	1.618	0.823
	36	**1.383**	**0.726**	1.502	0.770	1.808	0.870	1.624	0.801
	48	**1.482**	**0.776**	1.519	0.786	1.997	0.884	1.835	0.855
	60	**1.812**	**0.859**	1.881	0.895	2.028	0.896	1.601	0.829
	AVG	**1.550**	**0.783**	1.667	0.818	1.953	0.882	1.669	0.842
Traffic	96	**0.448**	**0.385**	0.483	0.386	0.533	0.430	0.467	0.390
	192	**0.478**	**0.393**	0.516	0.401	0.559	0.429	0.487	0.397
	336	**0.581**	**0.459**	0.610	0.495	0.625	0.498	0.603	0.488
	720	**0.647**	**0.593**	0.675	0.618	0.689	0.630	0.668	0.611
	AVG	**0.538**	**0.457**	0.571	0.475	0.602	0.497	0.556	0.472
Electricity	96	**0.208**	**0.285**	0.218	0.294	0.220	0.306	0.235	0.318
	192	**0.249**	**0.314**	0.253	0.305	0.254	0.320	0.279	0.340
	336	**0.489**	**0.370**	0.500	0.379	0.518	.394	0.539	0.415
	720	**0.865**	**0.439**	0.873	0.450	0.907	0.475	0.932	0.479
	AVG	**0.453**	**0.352**	0.461	0.357	0.475	0.374	0.496	0.388

6.5. Ablation study

To validate the effectiveness of the dual-level MLP framework for multivariate time series, we conducted ablation experiments on both modules: the Cross-Variable Block (CrossVBlock) and Intra-Variable Block (IntraVBlock), as shown in Table 8. Furthermore, to demonstrate the advantages of leveraging cross-variate information, we conducted ablation experiments on the CrossVBlock and DropFusion components separately, including replacing CrossVBlock with Attention (*_replace_Attention) and replacing DropFusion with Dropout (*_replace_Dropout), as shown in Tables 9 and 10. Comparative experiments revealed that incorporating CrossVBlock significantly improves forecasting performance, particularly compared to Attention mechanism, with an overall average improvement in MSE of **12.52%** and a maximum improvement of **32.65%**. This underperformance, in addition to Attention's limitation of capturing only linear multivariate relationships, may also be due to the fact that self-Attention necessitates linear weighting for pairs of variables, resulting in introducing extraneous noise to unrelated channels. Additionally, as shown in Table 10, experimental validation has confirmed the effectiveness of DropFusion, which outperforms Dropout at the same drop rate ($=0.5$), enabling better modeling of diverse relationships (the "D-DU" characteristic).

Overall, we observe that removing or replacing any component consistently degrades performance, with the degree of degradation varying depending on the strength of cross-variable dependencies across datasets. Notably, this variation is not directly correlated with the number of channels. For example, CrossVBlock plays a more critical role in datasets with strong cross-variable dependencies (e.g., Weather with 21 channels), whereas IntraVBlock is more important for capturing long-term temporal dependencies (e.g., Electricity, Traffic, and ETT), even in datasets with hundreds of variables such as Traffic and Electricity. In the following sections, we provide a more detailed analysis of the contributions of each component.

Cross-Variable Block and Intra-Variable Block. To validate the role of the Cross-Variable Block and Intra-Variable Block in capturing variable dependencies and temporal dependencies, we conducted ablation experiments by removing each module to test the forecasting accuracy of DyTimeNet. The specific experimental setups and results are shown in Table 8, which include the following configurations:

- **w/o_CrossVBlock**: No consideration of channel dependencies, assuming channels are independent, and only temporal dependencies are considered.
- **w/o_IntraVBlock**: Only channel dependencies are considered, with no consideration of temporal dependencies.
- **IntraVBlock → CrossVBlock**: Temporal dependencies are considered first, followed by channel information fusion.

Specifically, we conducted ablation studies on six representative datasets. As shown in Table 8, while the temporal module seems to have a more significant impact on the prediction results, it is clear that removing any module disrupts the model's optimal performance. These empirical findings highlight the indispensable role of both modules, which work in tandem to enhance each other and contribute to the overall effectiveness of the model. As shown in Table 8, removing either the cross-variate module (CrossVBlock) or the intra-variate temporal module (IntraVBlock) consistently degrades performance, confirming that both components contribute to DyTimeNet. Interestingly, the model's reliance on each module varies across datasets, which is closely related to the strength and nature of inter-channel dependencies. For example, on the Weather dataset, removing CrossVBlock causes a larger deterioration than removing IntraVBlock (AVG MSE:

Table 9
The effectiveness of cross-variable block vs. Attention mechanism. **IMP(↑)**: Improvement Percentage.

Component		*_w_CrossVBlock		*_replace_Attention		IMP(↑)	
dataset	pred_len	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	**0.310**	**0.352**	0.415	0.426	25.33%	17.40%
	192	**0.354**	**0.378**	0.425	0.425	16.64%	11.14%
	336	**0.387**	**0.401**	0.471	0.457	17.81%	12.28%
	720	**0.447**	**0.436**	0.526	0.487	15.07%	10.42%
	AVG	**0.374**	**0.392**	0.459	0.449	18.46%	12.72%
ETTm2	96	**0.172**	**0.254**	0.192	0.280	10.28%	9.26%
	192	**0.234**	**0.296**	0.253	0.313	7.64%	5.56%
	336	**0.296**	**0.335**	0.309	0.350	4.34%	4.23%
	720	**0.395**	**0.395**	0.408	0.405	3.26%	2.48%
	AVG	**0.274**	**0.320**	0.291	0.337	5.66%	5.06%
Weather	96	**0.155**	**0.202**	0.196	0.250	20.96%	19.04%
	192	**0.203**	**0.245**	0.247	0.288	17.78%	14.92%
	336	**0.250**	**0.286**	0.297	0.321	15.95%	10.76%
	720	**0.338**	**0.340**	0.370	0.368	8.62%	7.62%
	AVG	**0.237**	**0.268**	0.278	0.306	14.80%	12.48%
ILI	24	**1.522**	**0.771**	2.260	1.029	32.65%	25.03%
	36	**1.383**	**0.726**	1.536	0.848	9.93%	14.28%
	48	**1.482**	**0.776**	1.697	0.876	12.68%	11.44%
	60	**1.812**	**0.859**	2.117	0.998	14.41%	13.97%
	AVG	**1.550**	**0.783**	1.903	0.937	18.54%	16.48%
Traffic	96	**0.448**	**0.385**	0.514	0.420	12.94%	8.39%
	192	**0.478**	**0.393**	0.550	0.433	13.18%	9.19%
	336	**0.581**	**0.459**	0.618	0.501	5.99%	8.38%
	720	**0.647**	**0.593**	0.682	0.624	5.13%	4.97%
	AVG	**0.538**	**0.457**	0.591	0.494	8.93%	7.48%
Electricity	96	**0.208**	**0.285**	0.243	0.307	14.64%	7.24%
	192	**0.249**	**0.314**	0.270	0.324	7.76%	3.34%
	336	**0.489**	**0.370**	0.521	0.383	6.21%	3.49%
	720	**0.865**	**0.439**	0.899	0.456	3.70%	3.93%
	AVG	**0.453**	**0.352**	0.484	0.368	6.32%	4.38%
AVG_IMP(↑)				–		**12.52%**	**9.91%**

Table 10
The effectiveness of DropFusion for cross-variate dependencies.

Component		*_w_DropFusion		*_w/o_DropFusion		*_replace_Dropout	
dataset	pred_len	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	0.310	0.352	0.321	0.359	0.319	0.350
	192	0.354	0.378	0.375	0.391	0.352	0.376
	336	0.387	0.401	0.416	0.416	0.384	0.400
	720	0.447	0.436	0.480	0.453	0.452	0.439
	AVG	**0.374**	**0.392**	0.398	0.405	0.377	0.391
ETTm2	96	0.172	0.254	0.180	0.261	0.173	0.255
	192	0.234	0.296	0.245	0.302	0.244	0.302
	336	0.296	0.335	0.306	0.343	0.302	0.339
	720	0.395	0.395	0.406	0.400	0.402	0.393
	AVG	**0.274**	**0.320**	0.284	0.327	0.280	0.323
Weather	96	0.155	0.202	0.157	0.205	0.154	0.201
	192	0.203	0.245	0.206	0.250	0.204	0.248
	336	0.250	0.286	0.266	0.295	0.260	0.290
	720	0.338	0.340	0.349	0.347	0.342	0.342
	AVG	**0.237**	**0.268**	0.244	0.274	0.240	0.270
ILI	24	1.522	0.771	1.648	0.811	1.765	0.843
	36	1.383	0.726	1.444	0.742	1.351	0.728
	48	1.482	0.776	1.547	0.790	1.448	0.774
	60	1.812	0.859	1.884	0.889	1.692	0.841
	AVG	**1.550**	**0.783**	1.631	0.808	1.564	0.797
Traffic	96	**0.448**	**0.385**	0.452	0.405	0.462	0.404
	192	**0.478**	**0.393**	0.521	0.429	0.502	0.405
	336	**0.581**	**0.459**	0.597	0.476	0.606	0.482
	720	**0.647**	**0.593**	0.654	0.599	0.661	0.605
	AVG	**0.538**	**0.457**	0.556	0.477	0.558	0.474
Electricity	96	**0.208**	**0.285**	0.228	0.312	0.214	0.296
	192	**0.249**	**0.314**	0.268	0.330	0.255	0.319
	336	**0.489**	**0.370**	0.504	0.398	0.491	0.379
	720	**0.865**	**0.439**	0.857	0.432	0.864	0.930
	AVG	**0.453**	**0.352**	0.464	0.368	0.456	0.481

0.237 → 0.260 vs. 0.237 → 0.248), suggesting that Weather forecasting benefits more from modeling cross-channel interactions. This aligns with real-world meteorology scenarios, where variables such as temperature, pressure, humidity, and wind are strongly coupled, and forecasting a single variable often requires contextual information from other channels.

In contrast, ETTm1 relies more on intra-variate temporal modeling, with cross-channel relationships playing a complementary role (AVG MSE: 0.374 → 0.395 when removing IntraVBlock, compared to 0.374 → 0.385 when removing CrossVBlock). ETTm2, Traffic, Electricity and ILI show a similar tendency. In particular, ILI exhibits a substantial drop when removing IntraVBlock (AVG MSE: 1.550 → 1.953), indicating that accurate long-range forecasting in this dataset depends strongly on each channel's historical dynamics. This is mainly because the channels in these datasets are less tightly coupled in terms of physical semantics than those in Weather. For example, ETT records the temperatures of transformers at different sites, while ILI measures influenza-like illness counts across different states. In such cases, each channel is driven more by its own temporal evolution, and cross-channel interactions provide relatively limited additional information. Overall, these results demonstrate that DyTimeNet can flexibly exploit either cross-channel dependencies or intra-channel temporal patterns depending on the data characteristics, and further support the motivation of DropFusion to learn structured relationships in a data-adaptive manner.

Furthermore, to assess the impact of the ordering of the channel and temporal modules within the Time-Variable fusion module on predictive performance, we swapped the order of the modules **(IntraVBlock → CrossVBlock)**, starting with temporal processing followed by channel fusion. The results show that when temporal information extraction is followed by channel fusion, the overall predictive performance tends to be slightly worse. This may be due to the integrity of the temporal patterns being unintentionally disrupted during the channel interaction phase.

The Effectiveness of Cross-Variable Block. Furthermore, to examine whether CrossVBlock can uncover richer latent interactions among variables than a vanilla self-Attention module, we replace CrossVBlock with self-Attention (denoted as ***_replace_Attention**). As reported in Table 9, self-Attention is inferior to CrossVBlock on most datasets. More interestingly, Table 8 shows that ***_replace_Attention** even performs worse than directly removing CrossVBlock (***_w/o_CrossVBlock**). One plausible reason is that self-Attention mixes channels through dense pairwise weighting and aggregation, which can be sensitive to spurious pairwise affinities: when many channels are weakly related or conditionally independent, dense mixing may introduce unnecessary interference from irrelevant channels, thereby harming forecasting accuracy. In contrast, incorporating CrossVBlock yields consistent gains, with an average MSE improvement of **12.52%** and a maximum improvement of **32.65%** (Table 9). This confirms CrossVBlock's capability to effectively learn and exploit correlations between channels, thereby proving that integrating information from other channels substantially improves auxiliary predictions. We attribute this advantage to the proposed DropFusion strategy embedded CrossVBlock for channel relationship learning. By jointly applying neuron dropping and edge dropping, DropFusion effectively simulates diverse sparse network structures during training, enabling the model to adapt to complex channel relations (linear, nonlinear, and near-independent) and to learn more stable and informative representations.

DropFusion. DropFusion design allows the network to simulate diverse sparse and structured dependency patterns during training, where

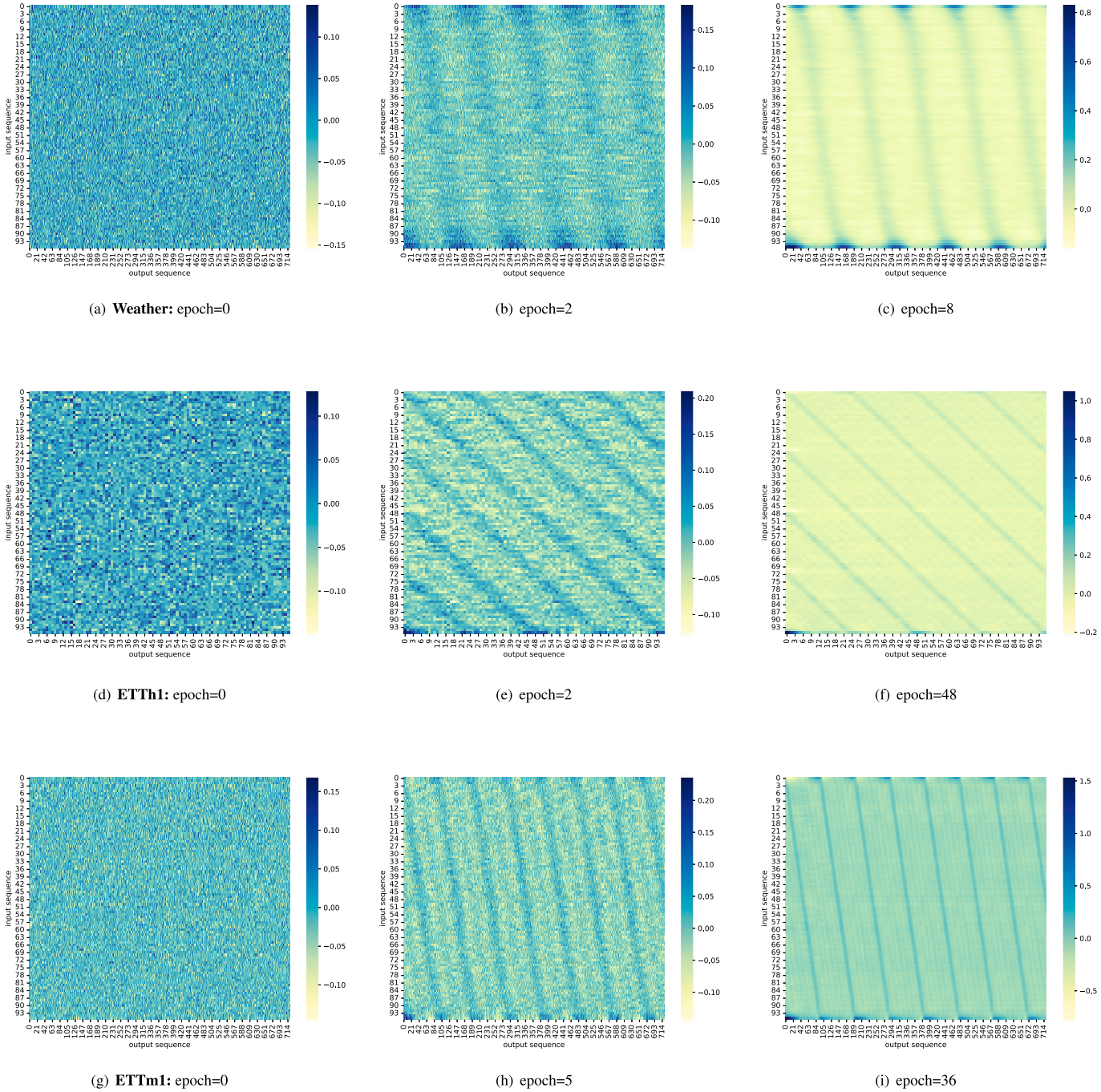
some variable pairs may be weakly related or independent, while others exhibit strong dependencies. Such structured sparsity serves as an inductive bias that encourages the model to focus on the most informative interactions and avoids overfitting to dense, potentially noisy channel mixing. As shown in Table 10, DropFusion consistently outperforms standard Dropout (***_replace_Dropout**) under the same drop rate (= 0.5). This is because Dropout primarily performs channel-level filtering and self-attention performs dense pairwise channel aggregation, neither explicitly enforcing learning of a meaningful channel structure. Therefore, both theoretical motivation and empirical results consistently demonstrate the advantage of DropFusion for modeling complex and dynamic multivariate relationships.

6.6. Model analysis

6.6.1. Temporal patterns learning capability

After training, we visualize the weights learned by the model to gain a deeper understanding of the learning process behind the predicted values. Based on the inherent characteristics of the data, we categorize the learning process into periodic, trend, and multi-periodic patterns, as shown in Figs. 11–13. These visualizations provide an intuitive representation of the neural network's learning process, transitioning from random noise/initialization to a stable specific distribution.

At the beginning of the training process (epoch = 0), the model's weights are typically initialized randomly, following some distribu-

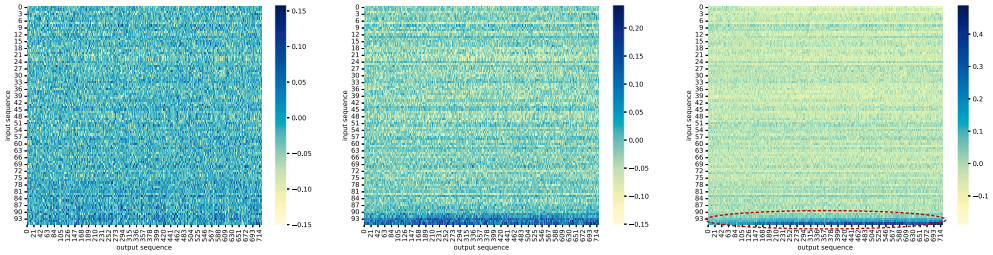


(a) **Weather:** epoch=0 (b) epoch=2 (c) epoch=8

(d) **ETTh1:** epoch=0 (e) epoch=2 (f) epoch=48

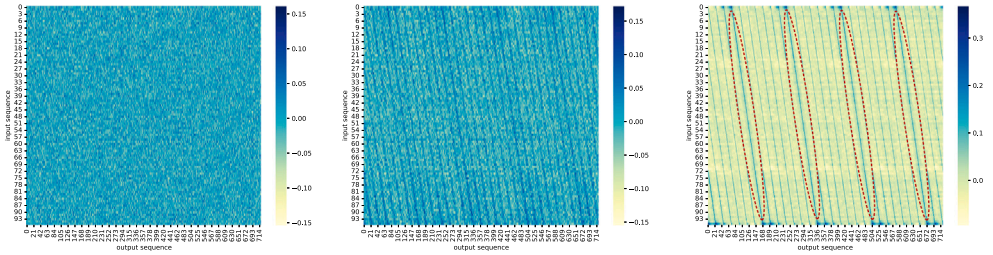
(g) **ETTm1:** epoch=0 (h) epoch=5 (i) epoch=36

Fig. 11. Visualization of model learning process of **periodic pattern**, demonstrated on the Weather, ETTh1, and ETTm1 datasets, starting from scratch. The lookback length (Y-axis) is set to 96. For ETTh1, the forecasting length (X-axis) is 96, while for the other datasets, it is set to 720, considering the sampling rate.

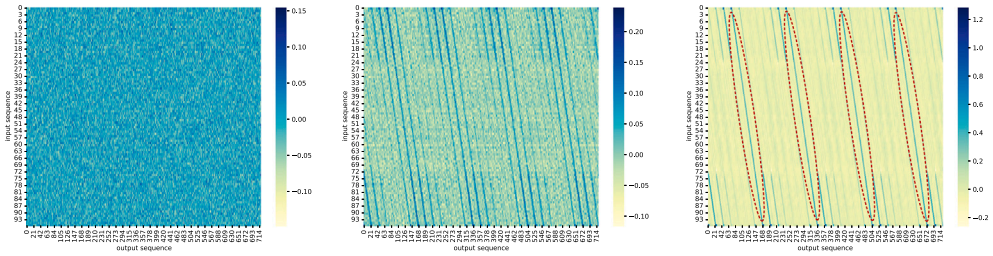


(a) **Exchange:** epoch=0 (b) epoch=2 (c) epoch=8

Fig. 12. Visualization of model learning process of **trend pattern**, demonstrated on the Exchange Rate dataset, starting from scratch. The lookback length (Y-axis) is set to 96 and forecasting (X-axis) lengths set to 720.



(a) **Electricity:** epoch=0 (b) epoch=2 (c) epoch=8



(d) **Traffic:** epoch=0 (e) epoch=2 (f) epoch=8

Fig. 13. Visualization of model learning process of **multi-periodic pattern**, demonstrated on the Exchange Rate dataset, starting from scratch. The lookback length (Y-axis) is set to 96 and forecasting (X-axis) lengths set to 720.

tion (e.g., Gaussian or uniform). At this stage, the network is in a "chaotic" state, with no clear patterns identified in the data. As training progresses (from epoch 2 to epoch 8), the model gradually converges, and the weight distribution becomes more stable, revealing various patterns in the time series, including periodicity, trends, and multi-periodicity. This indicates that the neural network, in the regression process, is able to capture not only the fundamental patterns in the time series but also learn finer-grained multi-periodic patterns, going beyond simple periodicity and trend patterns.

Specifically, in domains such as meteorology (Weather) and electricity (ETTh1, ETTm1) (Fig. 11), the model learns a clear 24-hour periodic pattern. For exchange rate data in the financial domain (Fig. 12), the model identifies a trend pattern, where the exchange rate for the next day is more dependent on the previous day's rate, with less influence from earlier exchange rates, which aligns with real-world behavior.

Additionally, in Traffic and Electricity load scenarios (ECL), as illustrated in Fig. 13, the model not only captures the usual 24-hour short cycles but also identifies longer periodic patterns, such as the 7-day (weekly) cycle. Over time, the model gradually uncovers and emphasizes the periodic patterns in the time series, assigning progressively higher weights to the most recent time steps within the lookback window. It is evident that higher weights are applied to forecasting steps at intervals like 0, 23, 47, …, and 95, which correspond to daily (24-hour) periods in the Electricity dataset, clearly reflecting its inherent daily periodicity. Moreover, the model assigns higher weights at 7-day intervals, such as 167, 335, 503, etc., confirming the weekly periodicity inherent in the Traffic dataset. By assigning higher weights to these periodic intervals, the model effectively captures both short-term and long-term variation trends in the data, which is especially important for datasets like traffic flow that exhibit multi-periodic features.

6.6.2. *Capacity to longer lookback sequence*

In time series forecasting, the ability to handle longer lookback sequences plays a critical role in capturing temporal dependencies over extended periods. As illustrated in Fig. 14, DyTimeNet demonstrates a superior capacity to model such dependencies, especially in scenarios where historical data spans across long intervals (with lookback window sizes (L) varying from 96 to 720). From the hourly-sampled ETTh1 to the minute-level ETTm1 and Weather datasets, it is evident that as the amount of historical information increases, DyTimeNet improves its performance for both short-term and long-term forecasting (with pred_len varying from 96 to 720). Specifically, for forecasting longer sequences with larger lookback window sizes (L), DyTimeNet shows significant improvements in performance metrics like Mean Squared Error (MSE). This is particularly evident in datasets with low-frequency minute-level sampling and inherent periodic patterns, such as

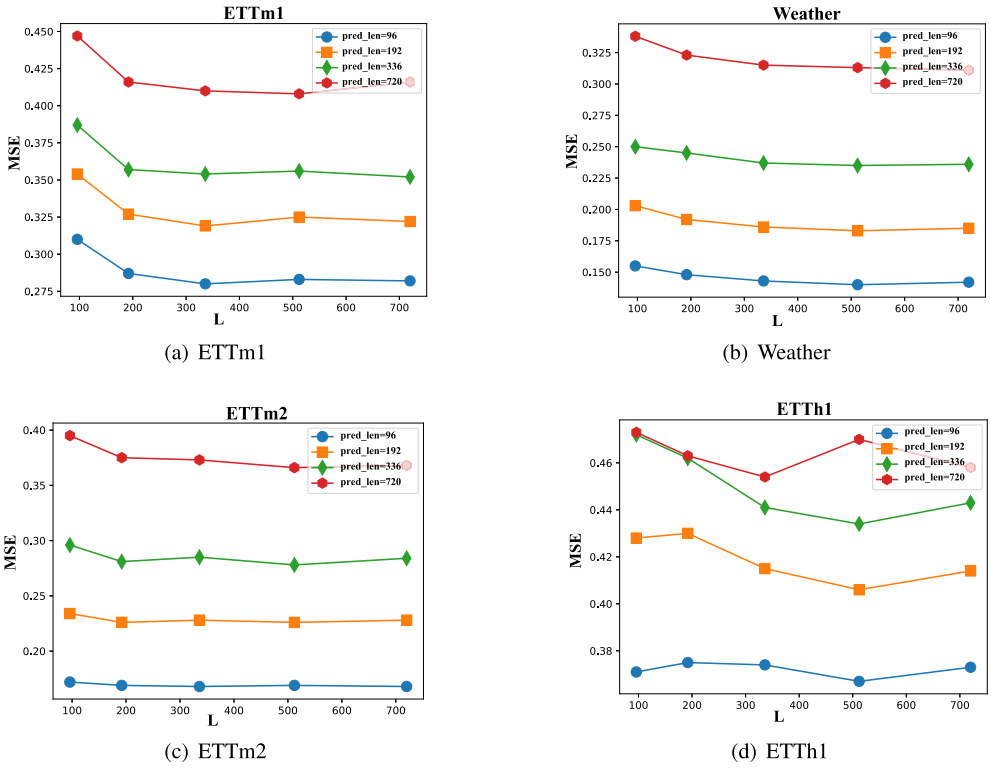


Fig. 14. The MSE results (Y-axis) of models with lookback window size (L) (X-axis) varying from {96, 192, 336, 512, 720}, and different prediction length (pred_len = {96, 192, 336, 720}). In most instances, prediction accuracy improves with a enlarged lookback length.

ETTm1, ETTm2, and Weather, where extended historical data enables DyTimeNet to better capture repeating cycles and long-term trends. By utilizing this additional historical context, the model is able to make more accurate predictions, especially for long-term forecasts. Unlike traditional Attention mechanisms in Transformer-based models, which often struggle with longer sequences due to computational complexity and temporal information loss [4,50], DyTimeNet's simpler yet effective architecture maintains training stability and achieves superior performance on longer sequences.

6.6.3. *Visualization of forecast results*

We perform a visual analysis by selecting MLP-based models (TimeMixer, DLinear) and Transformer-based models (iTransformer, PatchTST, Autoformer), focusing on two typical scenarios in stationary and non-stationary time series: sudden anomalies and stable periodic cycles. As demonstrated in Figs. 15 and 16, it is evident that DyTimeNet consistently outperforms other models in terms of prediction accuracy across datasets like Electricity and Traffic. In both cases, DyTimeNet produces predictions that closely align with the ground truth, showing minimal deviation between the predicted and actual values.

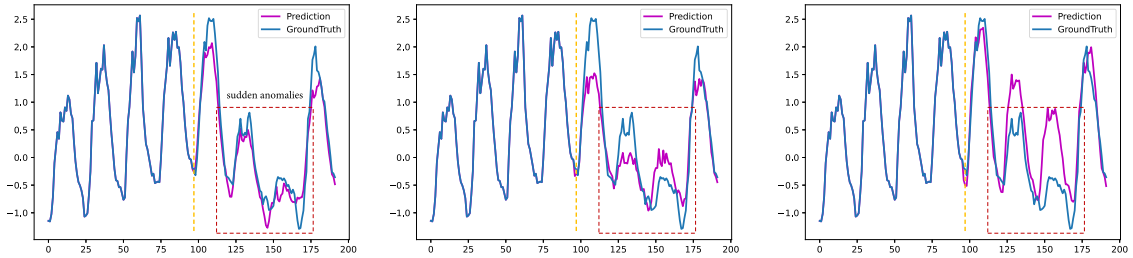
For the Electricity dataset (Fig. 15), the historical data exhibit stable periodic patterns, while the ground truth shows significant fluctuations between 125–175 steps. DyTimeNet effectively adapts to these sudden anomalies, accurately forecasting the trend changes. This is attributed to the model's ability to integrate relevant channel information, allowing it to anticipate future variations. In comparison, TimeMixer (Fig. 15b) shows limited sensitivity to anomalies, roughly detecting potential irregularities but struggling to predict peak values. iTransformer, PatchTST, and DLinear, however, exhibit weaker sensitivity to anomalies, continuing to forecast based on historical periodicity and missing the sudden changes.

In the Traffic dataset (Fig. 16), where periodicity is more prominent and represents a classic stationary time series, DyTimeNet outperforms other models with its finer-grained fitting. While other models capture the overall periodic pattern, they struggle with accurately predicting the peaks. Specifically, TimeMixer and iTransformer assume full correlation across all channels, which often leads to redundant information and poorer model fitting. As a result, these models exhibit significant biases, especially at the peaks. Additionally, PatchTST, Dlinear, and Autoformer, which employ the CI strategy and ignore inter-channel dependencies, exhibit noticeable biases at both peak and valley points, resulting in overall poorer fitting. Therefore, we believe that DyTimeNet's learning paradigm is better suited to capture the complex internal dependencies, resulting in superior prediction performance.

6.7. *Case study*

To clarify how our model learns channel relationships, especially under the proposed DropFusion strategy, we visualize the key training steps in this section. Specifically, we illustrate the two stochastic dropping operations used during training, namely DropEdge and DropNeuron, as shown in Fig. 17. Together, these operations encourage the model to form a sparse yet structured pattern of channel relationships.

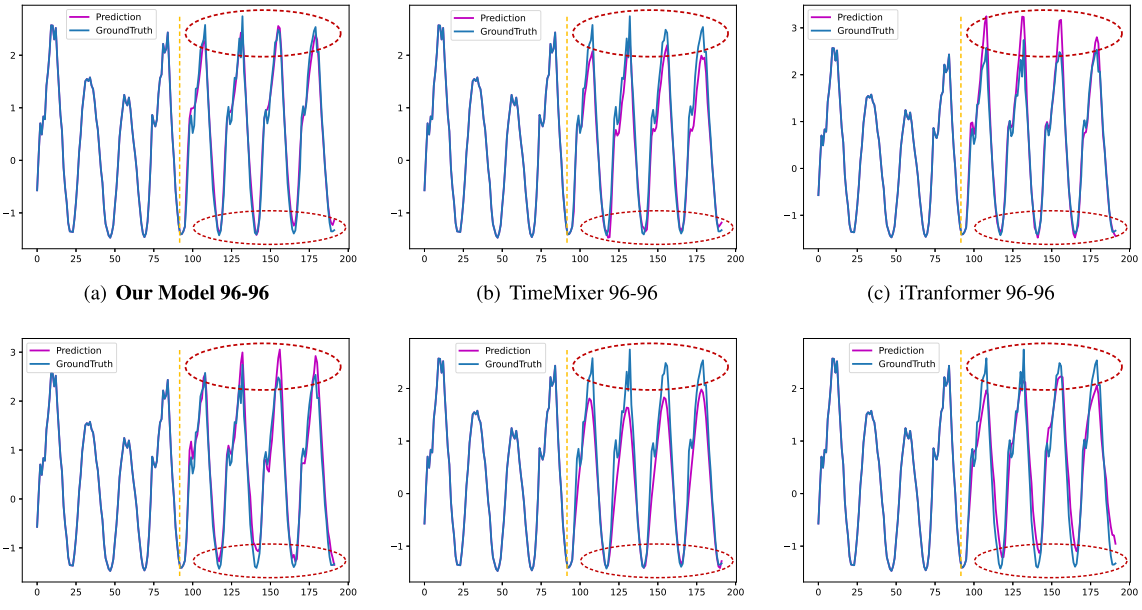
Compared with standard fully connected (dense) channel mixing, or applying neuron-level removal alone (i.e., Dropneuron/Dropout), DropFusion tends to produce sparse but structured channel relationship representations. Such representations better match real-world multivariate systems, where only a subset of variables interact and the interaction topology is rarely fully dense. Intuitively, Dropneuron promotes the suppression of entire channels that are largely independent or weakly informative (e.g., channel A or B), while Dropedge selectively removes uninformative pairwise channel connections. For example, if channel A is unrelated to channel B but both are related to channel C, Dropedge can drop the A-B connection while preserving A-C and B-C, thereby maintaining the meaningful structure of the underlying relationships. As a result, the learned connectivity becomes more interpretable and better aligned with the sparse, structured dependency patterns observed in practice, e.g., the meteorology example in Fig. 18(b).



(a) **Our Model 96-96** (b) TimeMixer 96-96 (c) iTranformer 96-96

(d) PatchTST 96-96 (e) Dlinear 96-96 (f) Autoformer 96-96

Fig. 15. Showcases the **sudden anomalies** in the input-96-predict-96 results on the Electricity dataset.



(a) **Our Model 96-96** (b) TimeMixer 96-96 (c) iTranformer 96-96

(d) PatchTST 96-96 (e) Dlinear 96-96 (f) Autoformer 96-96

Fig. 16. Showcases the **stable periodic cycles** in the input-96-predict-96 results on the Traffic dataset.

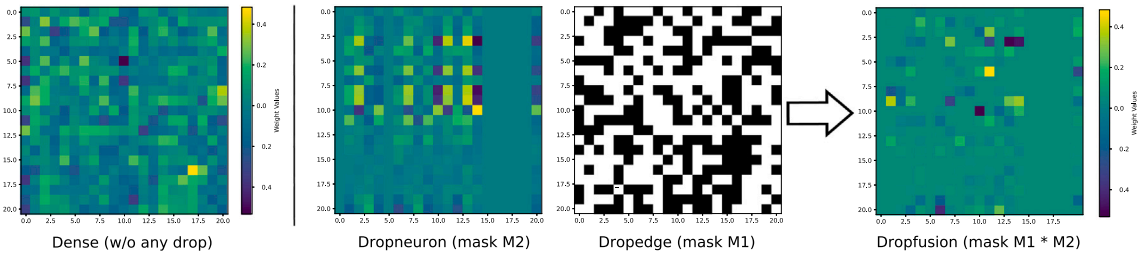
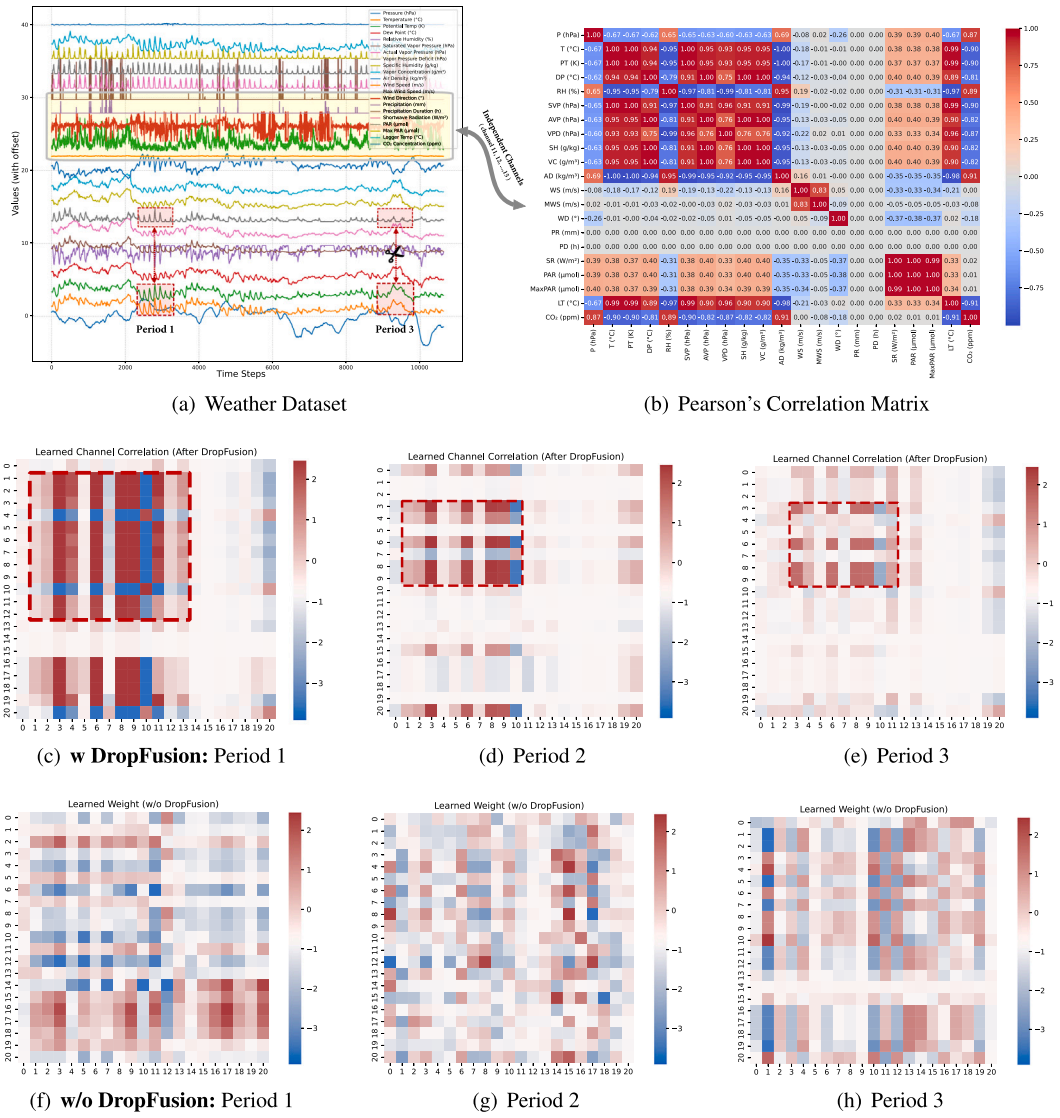


Fig. 17. DropFusion mechanism. Visualization of weight distributions under different drop strategies: dense (no drop), dropneuron (mask $M2$), dropedge (mask $M1$), and DropFusion (mask $M1 * M2$).



(a) Weather Dataset (b) Pearson's Correlation Matrix

(c) **w DropFusion:** Period 1 (d) Period 2 (e) Period 3

(f) **w/o DropFusion:** Period 1 (g) Period 2 (h) Period 3

Fig. 18. Learned dynamic channel relationships. (a) Multivariate Time series of weather domain across three periods. (b) Pearson's correlation matrix of 21 weather channels. Panels (c)–(e) show learned channel correlations with DropFusion for periods 1–3, while panels (f)–(h) show learned weights without DropFusion, highlighting the strategy's ability to capture structured, time-varying channel relationships.

Furthermore, the DropFusion strategy, by randomly dropping different neurons and connections across iterations, simulates various network structures. This enhances the diversity of the network's representation, enabling it to adapt to changing channel relationship patterns. Therefore, we visualize the channel relationship patterns learned by the model at different stages in the weather domain in Fig. 18. Panel (a) (Fig. 18a) shows the original weather dataset with 21 channels, and panel (b) (Fig. 18b) shows the overall Pearson correlation matrix of the channels in this dataset, which reflects linear dependence only. Two observations can be made: (1) Channels 11–15 show consistently weak correlations with other channels, suggesting that they are mostly independent or weakly related; (2) Channel relationships change subtly over time. For instance, channel 7 is linearly correlated with channels 1 and 2 in period 1, but shows no apparent linear relationship with them in period 3.

Based on these observations, we further investigate whether the model's learned relationships are consistent with the dataset's behavior. As shown in Fig. 18(c)–(e), by randomly selecting samples from three different periods and visualizing the inferred channel relationships, we observe that, across the three periods, the relationships between strongly correlated and near-independent channels remain consistent with Fig. 18(b). That is, the first 10 channels are strongly correlated, while channels 11 to 15 remain relatively independent. At the same time, the model captures subtle temporal differences across the three periods. In period 1, the inferred relationships are markedly stronger, while in period 2 and period 3, the relationships become weaker or more diffuse. This is consistent with the fact that meteorological dependencies may vary across different seasons or regimes, leading to stronger coupling in some periods and weaker or more complex interactions in others.

To further highlight the necessity of DropFusion for enabling neural networks to effectively fit and learn multivariate time series, Fig. 18(f)–(h) shows the channel relationships learned without any drop operations. With a fully connected network, the learned channel relationships become nearly uniform and fail to reflect the underlying structure. In contrast, by learning structured weight coefficients, our model can dynamically adapt to different inputs and capture a more diverse set of channel relationships. The resulting structured weight distributions can represent complex multivariate dependencies(including linear, nonlinear, and independent relationships) and effectively capture the combinations of multiple relationships.

7. Conclusion

In this paper, we propose DyTimeNet, a novel dual-level Feedforward Neural Network (FFN) model designed to enhance multivariate time series forecasting tasks. By introducing CrossVBlock and IntraVBlock, along with the DropFusion strategy, we address the challenge of capturing complex, dynamic, and sparse inter-variable dependencies. This enables the model to adapt to dynamic channel correlations while avoiding interference from irrelevant channels, leading to better fine-grained pattern fitting in multivariate time series. Moreover, extensive comparative results highlight DyTimeNet's strong predictive capability, particularly in handling datasets with periodic, trend-based, and even multi-periodic patterns. This significantly improves the model's ability to capture the complexities of time series data, resulting in more accurate predictions. Additionally, its ability to capture both short- and long-term dependencies makes it a robust choice for real-world applications, particularly for datasets like electricity load and traffic flow, which exhibit complex periodic behaviors. In terms of model performance, compared to cumbersome Transformer-based methods, DyTimeNet uses fewer parameters, trains faster, and delivers superior prediction results. This makes it especially valuable for practical applications, such as high-frequency prediction tasks or scenarios with limited computational resources. Therefore, we believe that DyTimeNet introduces a new paradigm in channel fusion for time series forecasting, particularly in tasks requiring efficient capture of long lookback sequences and dynamic dependencies. Its simplicity, scalability, and robustness make it a promising candidate for a wide range of future applications, from weather prediction to financial forecasting and traffic flow analysis.

CRediT authorship contribution statement

Ting Chen: Writing – original draft, Methodology, Investigation, Data curation, Conceptualization. **Jinzhou Lai:** Writing – review & editing, Visualization, Resources, Formal analysis. **Yining Sun:** Writing – review & editing, Validation, Data curation. **Wai Kin (Victor) Chan:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was funded by the National Natural Science Foundation of China under Grant 72574123. Additionally, this work was supported in part by the Major Key Project of PCL under Grant PCL2024A06 and PCL2025A12, and in part by the Shenzhen Science and Technology Program under Grant RCJC20231211085918010.

Data availability

I have share the link to my data in the paper.

References

[1] Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: long-term forecasting with transformers, in: The Eleventh International Conference on Learning Representations, 2023, https://openreview.net/forum?id=Jbdc0vTOcol.
[2] Z. Li, S. Qi, Y. Li, Z. Xu, Revisiting long-term time series forecasting: An investigation on linear mapping, arXiv preprint arXiv:2305.10721, 2023.
[3] T. Zhou, P. Niu, L. Sun, R. Jin, et al., One fits all: power general time series analysis by pretrained LM, Adv. Neural Inf. Process. Syst. 36 (2024).
[4] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting? in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, 2023, pp. 11121–11128.
[5] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, et al., Timemixer: decomposable multiscale mixing for time series forecasting, in: The Twelfth International Conference on Learning Representations, 2024, https://openreview.net/forum?id=7oLshfEIC2.
[6] S.-A. Chen, C.-L. Li, S.O. Arik, N.C. Yoder, T. Pfister, Tsmixer: an all-mlp architecture for time series forecast-ing, Trans. Mach. Learn. Res. 2023 (2023) ISSN 2835-8856.
[7] Z. Xu, A. Zeng, Q. Xu, FITS: modeling time series with $10k$ parameters, in: The Twelfth International Conference on Learning Representations, 2024, https://openreview.net/forum?id=bWcnvZ3qMb.
[8] M.M.N. Murad, M. Aktukmak, Y. Yilmaz, Wpmixer: efficient multi-resolution mixing for long-term time series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, 2025, pp. 19581–19588.
[9] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, et al., Itransformer: inverted transformers are effective for time series forecasting, in: The Twelfth International Conference on Learning Representations, 2023.
[10] X. Qiu, X. Wu, Y. Lin, C. Guo, J. Hu, B. Yang, Duet: dual clustering enhanced multivariate time series forecasting, in: SIGKDD, 2025, pp. 1185–1196.
[11] Y. Zhang, J. Yan, Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: The Eleventh International Conference on Learning Representations, 2023, https://openreview.net/forum?id=vSVLM2j9eie.
[12] Y. Wang, H. Wu, J. Dong, Y. Liu, Y. Qiu, H. Zhang, et al., Timexer: Empowering transformers for time series forecasting with exogenous variables, arXiv preprint arXiv:2402.19072, 2024.
[13] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, et al., Informer: beyond efficient transformer for long sequence time-series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, 2021, pp. 11106–11115.
[14] J. Ye, C. Zhou, X. Zhou, Y. Huang, C. Zhao, Cvacl-ma: comprehensive variate analysis and collaborative learning with multi-adapter for multivariate time series forecasting, Pattern Recognition 169 (2026) 111936.
[15] J. Ye, J. Li, C. Zhao, C. Zhou, X. Zhou, Y. Huang, Dual-stream interactive networks with pearson-mask awareness for multivariate time series forecasting, Neural Netw. (2025) 107588.
[16] T. Zhou, P. Niu, X. Wang, L. Sun, R. Jin, One fits all: power general time series analysis by pretrained LM, in: Thirty-Seventh Conference on Neural Information Processing Systems, 2023, https://openreview.net/forum?id=gMS6FVZvmF.
[17] X. Liu, J. Hu, Y. Li, S. Diao, Y. Liang, B. Hooi, et al., Unitime: a language-empowered unified model for cross-domain time series forecasting, in: Proceedings of the ACM on Web Conference 2024, 2024, pp. 4095–4106.
[18] A. Garza, M. Mergenthaler-Canseco, Timegpt-1, arXiv preprint arXiv:2310.03589, 2023.
[19] B.N. Oreshkin, D. Carpov, N. Chapados, Y. Bengio, N-beats: Neural basis expansion analysis for interpretable time series forecasting, arXiv preprint arXiv:1905.10437, 2019.
[20] T. Zhang, Y. Zhang, W. Cao, J. Bian, X. Yi, S. Zheng, et al., Less is more: Fast multivariate time series forecasting with light sampling-oriented mlp structures, arXiv preprint arXiv:2207.01186, 2022.
[21] C. Challu, K.G. Olivares, B.N. Oreshkin, F.G. Ramirez, M.M. Canseco, A. Dubrawski, Nhits: neural hierarchical interpolation for time series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, 2023, pp. 6989–6997.
[22] Z. Li, Z. Rao, L. Pan, Z. Xu, Mts-mixers: Multivariate time series forecasting via factorized temporal and channel mixing, arXiv preprint arXiv:2302.04501, 2023.
[23] V. Ekambaram, A. Jati, N. Nguyen, P. Sinthong, J. Kalagnanam, Tsmixer: lightweight mlp-mixer model for multivariate time series forecasting, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2023, pp. 459–469.
[24] I.O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, et al., Mlp-mixer: an all-mlp architecture for vision, Adv. Neural Inf. Process. Syst. 34 (2021) 24261–24272.
[25] L. Han, H.-J. Ye, D.-C. Zhan, The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting, arXiv preprint arXiv:2304.05206, 2023.
[26] L. Zhao, Y. Shen, Rethinking channel dependence for multivariate time series forecasting: learning from leading indicators, in: The Twelfth International Conference on Learning Representations, 2024, https://openreview.net/forum?id=JiTVtCUOpS.
[27] X. Qiu, J. Hu, L. Zhou, X. Wu, J. Du, B. Zhang, et al., Tfb: Towards comprehensive and fair benchmarking of time series forecasting methods, arXiv preprint arXiv:2403.20150, 2024.
[28] Q. Huang, L. Shen, R. Zhang, S. Ding, B. Wang, Z. Zhou, et al., Crossgnn: confronting noisy multivariate time series via cross interaction refinement, Adv. Neural Inf. Process. Syst. 36 (2023) 46885–46902.
[29] Y. Hu, G. Zhang, P. Liu, D. Lan, N. Li, D. Cheng, et al., Timefilter: patch-specific spatial-temporal graph filtration for time series forecasting, in: Forty-Second International Conference on Machine Learning, 2025, https://openreview.net/forum?id=490VcNtjh7.
[30] J.D. Hamilton, Time Series Analysis, Princeton University Press, 2020.
[31] X. Chen, C. Zhang, X. Chen, N. Saunier, L. Sun, Discovering dynamic patterns from spatiotemporal data with time-varying low-rank autoregression, IEEE Trans. Knowl. Data Eng. 36 (2) (2023) 504–517.
[32] K.D. Harris, A. Aravkin, R. Rao, B.W. Brunton, Time-varying autoregression with low-rank tensors, SIAM J. Appl. Dyn. Syst. 20 (4) (2021) 2335–2358.
[33] X. Chen, D. Zhuang, H. Cai, S. Wang, J. Zhao, Dynamic autoregressive tensor factorization for pattern discovery of spatiotemporal systems, IEEE Trans. Pattern Anal. Mach. Intell. 01 (2025) 1–14.
[34] R.M. French, Catastrophic forgetting in connectionist networks, Trends Cogn. Sci. 3 (4) (1999) 128–135.

[35] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A.A. Rusu, et al., Overcoming catastrophic forgetting in neural networks, Proc. Natl. Acad. Sci. 114 (13) (2017) 3521–3526.
[36] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, J. Choo, Reversible instance normalization for accurate time-series forecasting against distribution shift, in: International Conference on Learning Representations, 2021.
[37] M. Liu, A. Zeng, M. Chen, Z. Xu, Q. Lai, L. Ma, et al., Scinet: time series modeling and forecasting with sample convolution and interaction, Adv. Neural Inf. Process. Syst. 35 (2022) 5816–5828.
[38] Y. Liu, Q. Liu, J.-W. Zhang, H. Feng, Z. Wang, Z. Zhou, et al., Multivariate time-series forecasting with temporal polynomial graph neural networks, Adv. Neural Inf. Process. Syst. 35 (2022) 19414–19426.
[39] A. Das, W. Kong, A. Leach, S.K. Mathur, R. Sen, R. Yu, Long-term forecasting with tide: time-series dense encoder, Trans. Mach. Learn. Res. (2023) ISSN 2835-8856.
[40] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, Timesnet: temporal 2d-variation modeling for general time series analysis, in: International Conference on Learning Representations, 2023.
[41] C. Chen, K. Petty, A. Skabardonis, P. Varaiya, Z. Jia, Freeway performance measurement system: mining loop detector data, Transp. Res. Rec. 1748 (1) (2001) 96–102.
[42] R. Sen, H.-F. Yu, I.S. Dhillon, Think globally, act locally: a deep neural network approach to high-dimensional time series forecasting, Adv. Neural Inf. Process. Syst. 32 (2019).
[43] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: decomposition transformers with auto-correlation for long-term series forecasting, Adv. Neural Inf. Process. Syst. 34 (2021) 22419–22430.
[44] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, Fedformer: frequency enhanced decomposed transformer for long-term series forecasting, in: International Conference on Machine Learning, PMLR, 2022, pp. 27268–27286.
[45] Y. Liu, H. Wu, J. Wang, M. Long, Non-stationary transformers: exploring the stationarity in time series forecasting, Adv. Neural Inf. Process. Syst. 35 (2022) 9881–9893.
[46] M. Goswami, K. Szafer, A. Choudhry, Y. Cai, S. Li, A. Dubrawski, Moment: A family of open time-series foundation models, arXiv preprint arXiv:2402.03885, 2024.
[47] Z. Wang, F. Kong, S. Feng, M. Wang, H. Zhao, D. Wang, et al., Is mamba effective for time series forecasting? arXiv preprint arXiv:2403.11144, 2024.
[48] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, D. Sahoo, Unified training of universal time series forecasting transformers, in: Forty-First International Conference on Machine Learning, 2024, https://openreview.net/forum?id=Yd8eHMY1wz.
[49] A. Gu, T. Dao, Mamba: Linear-time sequence modeling with selective state spaces, arXiv preprint arXiv:2312.00752, 2023.
[50] M. Tan, M. Merrill, V. Gupta, T. Althoff, T. Hartvigsen, Are language models actually useful for time series forecasting? Adv. Neural Inf. Process. Syst. 37 (2024) 60162–60191.

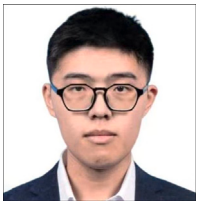
Author biography



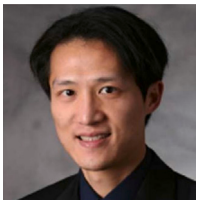
Ting Chen is currently pursuing the Ph.D. degree with the Tsinghua Shenzhen International Graduation School (SIGS), Tsinghua University, China. Her research interests include multivariate time series, deep learning, and representation learning.



Jinzhou Lai is currently pursuing the Eng.D. degree in Electrical and Electronic Engineering with The Hong Kong Polytechnic University, China. His research interests include smart analysis and intelligent inspection.



Yining Sun is currently pursuing the Ph.D. degree in Electronic Information Engineering with the Tsinghua Shenzhen International Graduation School (SIGS), Tsinghua University, China. His research interests include operations research and data science.



Wai Kin (Victor) Chan received the PhD degree in industrial engineering and operations research from the University of California, Berkeley. He is professor of the Shenzhen International Graduation School (SIGS), Tsinghua University, China. His research interests include agent-based and discrete-event simulation, data science and operations research, and their applications in emergency management, edverse, social networks, service systems, healthcare, transportation, energy markets, and manufacturing.